Lab_1

March 8, 2026

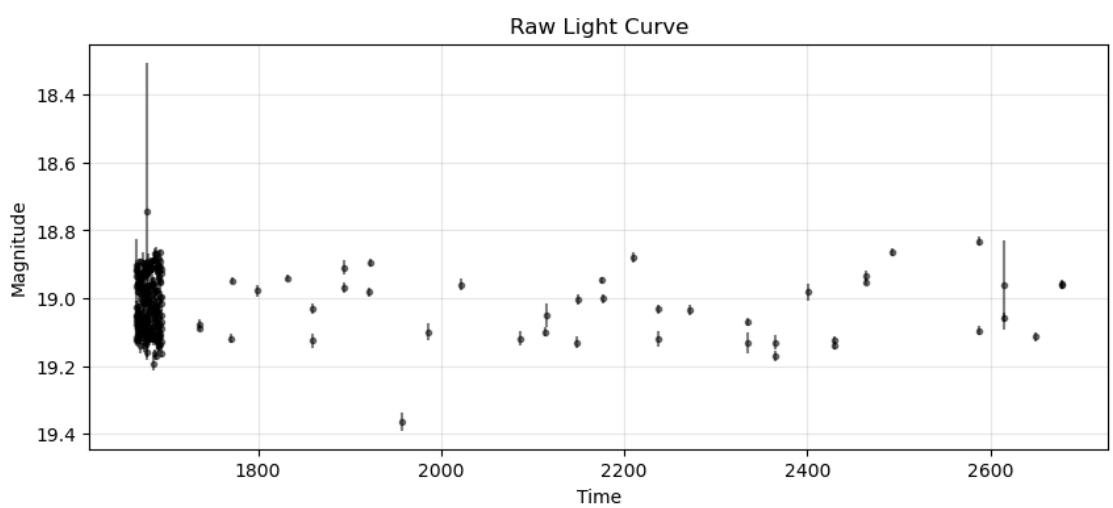
[1]:
```
from astroquery.gaia import Gaia
import pandas as pd
#query gaia data
query = """
SELECT TOP 100 *
FROM gaiadr3.vari_rrlyrae
WHERE pf IS NOT NULL
  AND num_clean_epochs_g > 40
ORDER BY num_clean_epochs_g DESC
"""

job = Gaia.launch_job_async(query)#asynchronous to not overload server
rrl_results = job.get_results()
df_rrl = rrl_results.to_pandas()#convert to pandas df
source_ids = df_rrl['source_id'].tolist()#get list of source ids

light_curves = {}

# I was getting an Error 500 from overloading the server, so I fixed it by␣
↪creating a loop that queries 1 light curve at a time and then appends it to␣
↪the pandas dataframe
for i, source_id in enumerate(source_ids):
    try:
        # distinct=True is required to get the specific light curve
        datalink = Gaia.load_data(ids=[source_id], data_release='Gaia DR3',
                                  retrieval_type='EPOCH_PHOTOMETRY',
                                  format='votable')

        # i only need the first table from the dictionary it returns
        dl_keys = list(datalink.keys())
        if len(dl_keys) > 0:
            lc_table = datalink[dl_keys[0]][0].to_table() # Convert to Astropy␣
↪Table
            light_curves[source_id] = lc_table.to_pandas() # Store as Pandas

    except Exception as e:
        print(f"Couldn't retrieve light curves")
```

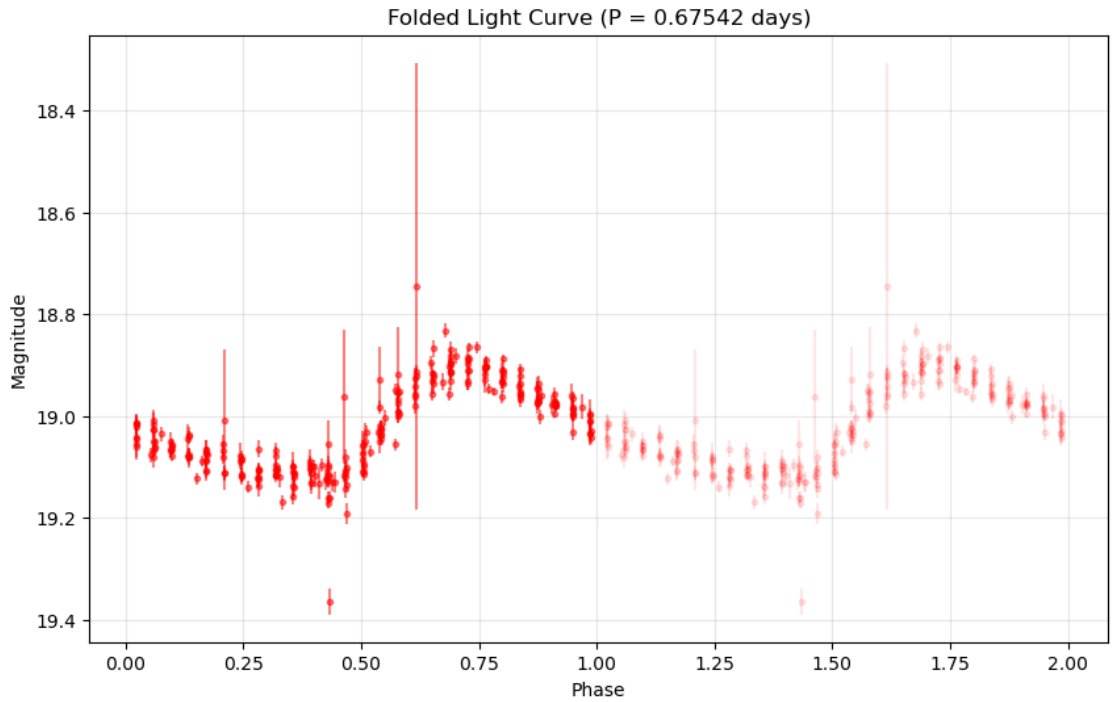
```
print(f"Downloaded {len(light_curves)} light curves.")
```

```
Some IP addresses of users launching heavy query showers have temporarily been
disabled. Please contact the Gaia helpdesk
(https://www.cosmos.esa.int/web/gaia/gaia-helpdesk) for advice. Workaround
solutions for the Gaia Archive issues following the infrastructure upgrade:
https://www.cosmos.esa.int/web/gaia/news#WorkaroundArchive
INFO: Query finished. [astroquery.utils.tap.core]
Downloaded 100 light curves.
```

[2]:
```
import matplotlib.pyplot as plt
import numpy as np

#  selecting first star from table
target_index = 0
star_row = df_rrl.iloc[target_index]
source_id = star_row['source_id']

period = star_row['pf'] #gaia period
#i thought pf stood for pulsation frequency, but apparently its actually in␣
↪units of time, so initially i calculated my period as 1/pf
# and i kept getting the wrong values, lowkey such a pain, but i figured it out␣
↪so no more complaints

lc_data = light_curves[source_id] #retrieving lc

#i pulled the whole table from gaia db so now im cleaning the g_band data
g_band_data = lc_data[['g_transit_time', 'g_transit_mag',␣
↪'g_transit_flux_over_error']].dropna()
g_band_data = g_band_data.rename(columns={
    'g_transit_time': 'time',
    'g_transit_mag': 'mag',
    'g_transit_flux_over_error': 'flux_over_error'
})

# magnitude  error derived from flux/error ratio provided by gaia
# i took the derivative of magnitude with respect to flux to derive the 1.0857␣
↪term
# (d/dF)m= (d/dF)-2.5log10*(F)+C -> 2.5/ln(10) = 1.0857
g_band_data['error'] = 1.0857 / g_band_data['flux_over_error']

# Phase = (Time / Period) - IntegerPart(Time / Period)
g_band_data['phase'] = (g_band_data['time'] % period) / period

# Raw light curve plot
plt.figure(figsize=(10, 4))
```

```
plt.errorbar(g_band_data['time'], g_band_data['mag'],
             yerr=g_band_data['error'],
             fmt='.', color='black', alpha=0.5, label='G-Band')

plt.gca().invert_yaxis() # inverting y axis b/c brighter stars have lower␣
 ↪magnitdues (thanks hipparchus/contemporary astronomers)
plt.title(f"Raw Light Curve")
plt.xlabel("Time")
plt.ylabel("Magnitude")
plt.grid(True, alpha=0.3)
plt.show()

# Phase folded LC
plt.figure(figsize=(10, 6))

# i plotted 2 cycles because it created a more visually appealing/digestable␣
 ↪graph
plt.errorbar(g_band_data['phase'], g_band_data['mag'],
             yerr=g_band_data['error'], fmt='.', color='red', alpha=0.5)

plt.errorbar(g_band_data['phase'] + 1.0, g_band_data['mag'],
             yerr=g_band_data['error'], fmt='.', color='red', alpha=0.1)

plt.gca().invert_yaxis()
plt.title(f"Folded Light Curve (P = {period:.5f} days)")
plt.xlabel("Phase")
plt.ylabel("Magnitude")
plt.grid(True, alpha=0.3)
plt.show()
```

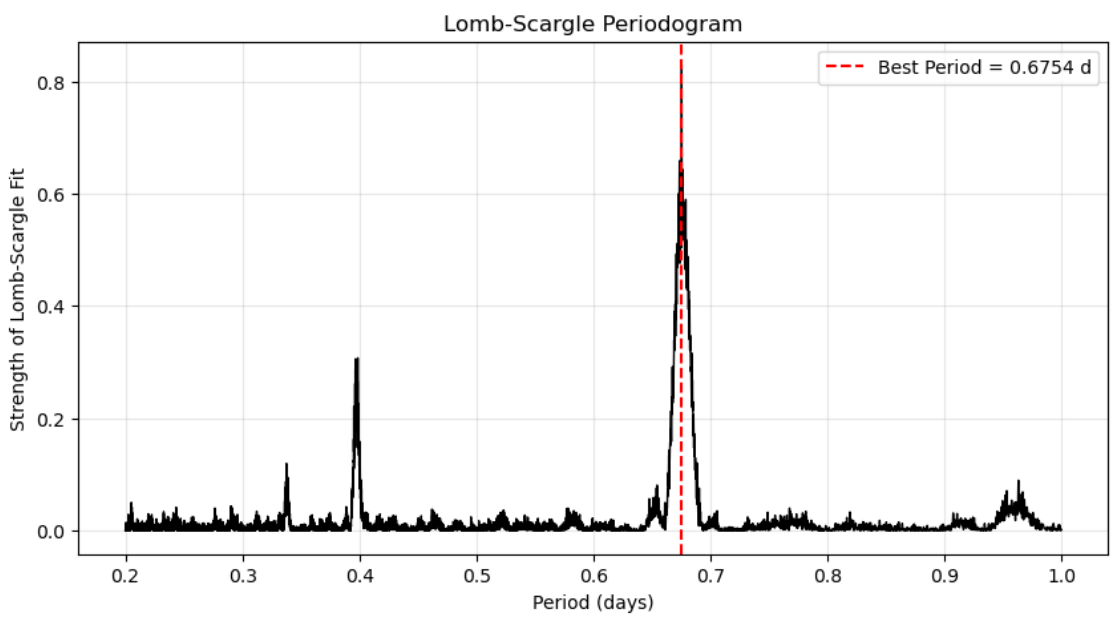




[3]:
```
from astropy.timeseries import LombScargle

#Lomb Scargle Periodigrams (goofy name lol) are a method of estimating the␣
↪period of an oscillating data set.
#essentially they try to fit multiple sine waves of varying frequencies to the␣
↪data set
# the sine wave that tracks the best (minimizes variance) is considered the␣
↪best fit. You can then use the period of this sine wave to approximate teh␣
↪period of your data!
# the periodogram is plotted w/ sin freq on x-axis and power (how good that␣
↪sine wave fit to the data) on the y-axis

#frequency grid
min_freq = 1.0 / 1.0  # 1 day
max_freq = 1.0 / 0.2  # 0.2 days

# Lomb-Scargle Periodogram
# parameters are time (t) magnitude (y), and  the Error (dy)
frequency, power = LombScargle(g_band_data['time'],
                               g_band_data['mag'],
                               g_band_data['error']).
↪autopower(minimum_frequency=min_freq,
```

```
 ↪maximum_frequency=max_freq)

# finding which sine wave fits the data the best
best_freq_index = np.argmax(power)
best_freq = frequency[best_freq_index]
best_period = 1.0 / best_freq

print(f"Gaia Period: {period:.5f} days")
print(f"Lomb-Scargle Estimated Period: {best_period:.5f} days")

# Plot
plt.figure(figsize=(10, 5))
plt.plot(1.0 / frequency, power, color='black', lw=1)

# Marking the best peak
plt.axvline(best_period, color='red', linestyle='--', label=f'Best Period =␣
 ↪{best_period:.4f} d')

plt.title(f"Lomb-Scargle Periodogram")
plt.xlabel("Period (days)")
plt.ylabel("Strength of Lomb-Scargle Fit")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
Gaia Period: 0.67542 days
Lomb-Scargle Estimated Period: 0.67544 days
```



[4]:
```
# 'g_band_data' contains light curve with 'mag' and 'flux_over_error'

# converting magnitude to flux in order to get an accurate calculated mean␣
↪since finding the mean using magnitudes would be innacurate due to the␣
↪logarithmic nature of the quantity
# m = -2.5 * log10(F)  => inverting log func. w/ exp base 10 => F = 10^(-0.4 *␣
↪m)
g_band_data['flux_arbitrary'] = 10**(-0.4 * g_band_data['mag'])

#using numpy to find mean flux
mean_flux = np.mean(g_band_data['flux_arbitrary'])

# taking log of mean flux gives accurate mean magnitude
estimated_mean_mag = -2.5 * np.log10(mean_flux)

# comparing with simple mean for fun
simple_mean_mag = np.mean(g_band_data['mag'])

#comparing w/ gaia official value
gaia_mean = star_row['int_average_g']

print(f"Simple Mean:           {simple_mean_mag:.4f}")
print(f"Flux-Weighted Mean:    {estimated_mean_mag:.4f}")
print(f"Gaia Value:            {gaia_mean:.4f}")
print(f"Difference:            {estimated_mean_mag - gaia_mean:.4f} mag")
```

```
Simple Mean:           19.0228
Flux-Weighted Mean:    19.0194
Gaia Value:            19.0215
Difference:            -0.0020 mag
```

[5]:
```
#it seems like the simple mean is closer to the gaia value. interesting
```

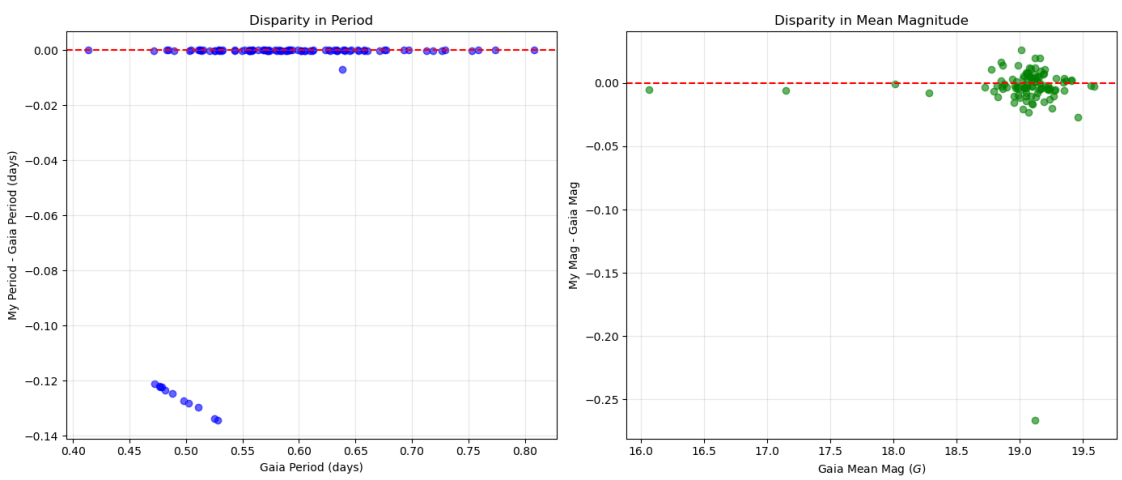
[6]:
```
results_list = [] # initializing a list to store the results

print(f"Processing {len(df_rrl)} stars...")

for index, row in df_rrl.iterrows():
    source_id = row['source_id']

    # pulling gaia values
    gaia_period = row['pf']
    gaia_mag = row['int_average_g'] # RR Lyrae mean mag column

    # retrieving light curve data
    if source_id in light_curves:
```

```
        lc = light_curves[source_id]

        # filtering for g-band
        # added a check to make sure the column exists because i kept pulling␣
↪the wrong/nonexistent data
        if 'g_transit_time' in lc.columns and 'g_transit_mag' in lc.columns:
            g_data = lc[['g_transit_time', 'g_transit_mag',␣
↪'g_transit_flux_over_error']].dropna()

            t = g_data['g_transit_time'].values
            y = g_data['g_transit_mag'].values
            dy = 1.0857 / g_data['g_transit_flux_over_error'].values # Approx␣
↪error

            # finding the period w/ lomb scargle ---
            # A good search range for RR Lyrae variables is 0.2 to 1.0 days␣
↪(source: https://ogle.astrouw.edu.pl/atlas/RR_Lyr.html)
            frequency, power = LombScargle(t, y, dy).
↪autopower(minimum_frequency=1.0,
                                                                    ␣
↪maximum_frequency=5.0)
            best_freq = frequency[np.argmax(power)]
            my_period = 1.0 / best_freq

            # --- calculating mean magnitude w. weighted flux
            flux_arbitrary = 10**(-0.4 * y)
            mean_flux = np.mean(flux_arbitrary)
            my_mag = -2.5 * np.log10(mean_flux)

            # Store everything
            results_list.append({
                'source_id': source_id,
                'gaia_period': gaia_period,
                'my_period': my_period,
                'gaia_mag': gaia_mag,
                'my_mag': my_mag
            })

# Convert to DataFrame for easy plotting
results_df = pd.DataFrame(results_list)
print(f"Done! Analyzed {len(results_df)} stars.")
```

```
Processing 100 stars…
Done! Analyzed 100 stars.
```

[7]:
```
fig, ax = plt.subplots(1, 2, figsize=(14, 6))
```

```python
# period comparison plot
# plot the Difference (My Period - Gaia Period) vs Gaia Period
period_diff = results_df['my_period'] - results_df['gaia_period']

ax[0].scatter(results_df['gaia_period'], period_diff, color='blue', alpha=0.6)
ax[0].axhline(0, color='red', linestyle='--') # The "Perfect Match" line
ax[0].set_title('Disparity in Period')
ax[0].set_xlabel('Gaia Period (days)')
ax[0].set_ylabel('My Period - Gaia Period (days)')
ax[0].grid(True, alpha=0.3)

# magnitude comparison
# plotting (My Mag - Gaia Mag) vs Gaia Mag
mag_diff = results_df['my_mag'] - results_df['gaia_mag']

ax[1].scatter(results_df['gaia_mag'], mag_diff, color='green', alpha=0.6)
ax[1].axhline(0, color='red', linestyle='--')
ax[1].set_title('Disparity in Mean Magnitude')
ax[1].set_xlabel('Gaia Mean Mag ($G$)')
ax[1].set_ylabel('My Mag - Gaia Mag')
ax[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Print stats
print(f"Mean Period Difference: {np.mean(period_diff):.6f} days")
print(f"Mean Magnitude Difference: {np.mean(mag_diff):.4f} mag")
```




```
Mean Period Difference: -0.013957 days
Mean Magnitude Difference: -0.0040 mag
```

0.0.1 Some Comments on the Data

- **Period:** The period is matched very well, with the exception of the strip of stars that do not adhere to the fit, with a period of ~ 0.5 days. Over an interval of ~.1 days the Gaia data is off by a margin of ~.125 days. I have two theories for why my data includes the divergent strip.
  - Hypothesis 1: The OGLE Atlas of Variable Star Light Curves has a very educational page that details the different types of RR Lyrae stars and how they behave. The website discusses the Blazhko effect. The Blazhko effect describes a curious characteristic of RR Lyrae stars; their period and amplitude change periodically! So over the course of one cycle, you will see a variable star shorten its period, and increase its brightness, and then return to normal. This could account for why some of the stars are deviating from the best fit period.
  - Hypothesis 2: This is the theory that I believe to be more likely. The source cited above describes that there are multiple "pulsation modes" for RR Lyrae stars. The primary mode RRab stars) occurs on a period of 0.2 to 1.0 days, however there is a second mode, for stars (called RRc stars) with periods between 0.2 and 0.5 days. If Gaia happened to accidentally catalog some RRc stars as RRab stars, then it would explain the trend seen very well, especially since their period is within the bounds observed for RRc stars. Not to mention the linear trend insinuates that there is a related nature to the divergence of these stars from the main strip; such as having a consistent period that oscillates due to a different mechanism than RRab stars.
- **Magnitude:** The magnitude is matched well with a clear standard deviation for the matching of the data sets. My theory as to why the magnitude may diverge is because of the Blazhko effect described earlier. Since the stars' magnitude varies periodically, but the period in which it varies periodically, it is possible that the data for these stars deviates from whatever the current calculated mean is, yet they still deviate with a consistent manner due to the undulating nature of the phenomena.

0.0.2 Fourier Analysis

Since the period of the stars is known, approximating the stars fluxes with some function f(t) can be done with a fourier series.

```
f(t) = A_0 + SUM[a_n * cos(nwt) + b_n * sin(nwt)]
```

This can be reduced to a linear system of equations since w is a known quantity (found w/ the periodogram).

y = X B

Y is a column vector consisting of the observed fluxes. X is a matrix that consists of the known terms in the fourier series (1, cos(nwt_initial), sin(wt_initial)). B is a column vector consisting of the unknown coefficients A_0, a_n, and b_n.

This system of equations allows us to solve for B pretty easily (with a computer, haha). Multiplying both sides by X_transpose makes it so B is forming an inner product with a square matrix. Since (X_transpose X) is a square matrix it is also invertible.

Therefore B (and all the coefficients to the series) is given by : B = (X_transpose X)^-1 X_transpose y

Luckily numpy has functions to do this for me!!!

[8]:
```python
import numpy as np

def create_design_matrix(t, period, K):
    """
    Parameters:
    - t: Array of observation times
    - period: The period of the star
    - K: The iteration of the series summation

    Returns X; the design matrix with shape (N, 2K+1)
    """
    # calculate w
    omega = 2 * np.pi / period

    # creating X matrix
    # column of 1's to leave A_0 unchanged
    # 2 columns (sin, cos) for each K
    n_rows = len(t)
    n_cols = 1 + 2 * K
    X = np.ones((n_rows, n_cols))

    # Fill the columns
    for k in range(1, K + 1):
        # k=1 -> cols 1,2; k=2 -> cols 3,4; etc.
        sin_col = 2 * k - 1
        cos_col = 2 * k

        X[:, sin_col] = np.sin(k * omega * t)
        X[:, cos_col] = np.cos(k * omega * t)

    return X
```

0.0.3 Function usage:

1. Get data for the specific star ex: Star ID: 4659713442253931776 (star for the next part)

target_id = 4659713442253931776 lc = light_curves[target_id] t_obs = … m_obs = … period = … (from Gaia or Lomb-Scargle)

2. Solve for Beta (Example for K=3) K = 3 X = create_design_matrix(t_obs, period, K) beta, residuals, rank, s = np.linalg.lstsq(X, m_obs, rcond=None)

3. Predict the Model model_mags = X @ beta

[9]:
```python
from astroquery.gaia import Gaia

# target ID for the star to analyze
target_id = 4659713442253931776 #star specified in the assgn.
```

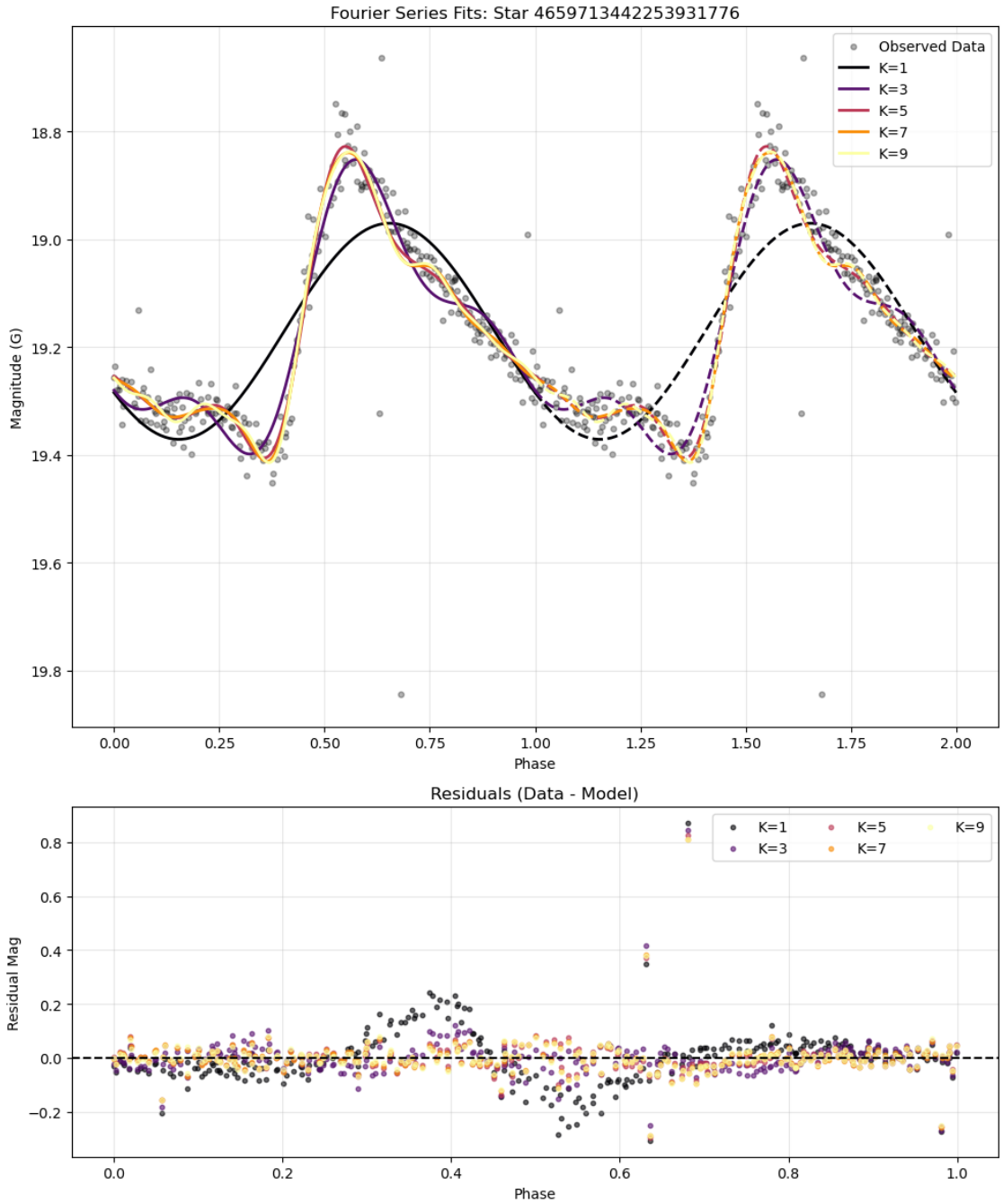
```
# check to see if its already downloaded
if target_id in light_curves:
    print(f"Star {target_id} is already in library!")
    lc_target = light_curves[target_id]
else:
    print(f"Downloading data for star {target_id}...")
    # Fetch the light curve
    datalink = Gaia.load_data(ids=[target_id], data_release='Gaia DR3',
                              retrieval_type='EPOCH_PHOTOMETRY',
                              format='votable')

    # Extract the table
    for key in datalink.keys():
        table = datalink[key][0].to_table().to_pandas()
        if 'mag' in table.columns or 'g_transit_mag' in table.columns: #␣
 ↪iterate to find the table with 'mag' columns
            lc_target = table
            break

# clean & prep data (g-band)
if 'g_transit_mag' in lc_target.columns:
    # Wide format
    g_data = lc_target[['g_transit_time', 'g_transit_mag',␣
 ↪'g_transit_flux_over_error']].dropna()
    g_data.columns = ['time', 'mag', 'flux_over_error']
else:
    # Long format
    g_data = lc_target[lc_target['band'] == 'G'][['time', 'mag',␣
 ↪'flux_over_error']].dropna()

# Get the period from 'vari_rrlyrae' column pf
query = f"SELECT pf FROM gaiadr3.vari_rrlyrae WHERE source_id = {target_id}"
job = Gaia.launch_job(query)
res = job.get_results()
period = res['pf'][0]

# Calculate Phase
g_data['phase'] = (g_data['time'] % period) / period

print(f"Ready to fit Star {target_id} (P = {period:.5f} d)")
```

```
Star 4659713442253931776 is already in library!
Ready to fit Star 4659713442253931776 (P = 0.60501 d)
```

[10]:
```
# prepare plotting

K_values = [1, 3, 5, 7, 9]
```

```
colors = plt.cm.inferno(np.linspace(0, 1, len(K_values)))

fig, axes = plt.subplots(2, 1, figsize=(10, 12), gridspec_kw={'height_ratios':␣
↪[2, 1]})
ax_fit = axes[0]
ax_res = axes[1]

# plotting phase folded raw data
ax_fit.scatter(g_data['phase'], g_data['mag'], color='black', alpha=0.3, s=15,␣
↪label='Observed Data')
ax_fit.scatter(g_data['phase']+1, g_data['mag'], color='black', alpha=0.3,␣
↪s=15) #adding a second phase just to better see the trend

# Loop through K values
for K, color in zip(K_values, colors):

    # Build Matrix X for Observed Time Points
    X_obs = create_design_matrix(g_data['time'].values, period, K)

    # Solving the y = X * beta system of eqn using least squares
    # rcond=None handles singularity warnings
    beta, residuals, rank, s = np.linalg.lstsq(X_obs, g_data['mag'].values,␣
↪rcond=None)

    # Create smooth model for plotting
    # Generate a grid of phases 0 to 1
    phase_grid = np.linspace(0, 1, 200)
    time_grid = phase_grid * period # Convert phase back to time units for the␣
↪function

    X_grid = create_design_matrix(time_grid, period, K)
    mag_model = X_grid @ beta # Matrix Multiplication

    # Plot
    ax_fit.plot(phase_grid, mag_model, color=color, lw=2, label=f'K={K}')
    ax_fit.plot(phase_grid+1, mag_model, color=color, lw=2, linestyle='--') #␣
↪Plot copy

    # Calculate and plot residuals (Data - Model)
    # the residuals are found using what the model predicted exactly AT the␣
↪observation times
    model_at_obs = X_obs @ beta
    resid = g_data['mag'] - model_at_obs
    ax_res.scatter(g_data['phase'], resid, color=color, s=10, alpha=0.6,␣
↪label=f'K={K}')
```

```python
# formatting
ax_fit.invert_yaxis()
ax_fit.set_title(f'Fourier Series Fits: Star {target_id}')
ax_fit.set_ylabel('Magnitude (G)')
ax_fit.set_xlabel('Phase')
ax_fit.legend()
ax_fit.grid(True, alpha=0.3)

ax_res.axhline(0, color='black', linestyle='--')
ax_res.set_title('Residuals (Data - Model)')
ax_res.set_ylabel('Residual Mag')
ax_res.set_xlabel('Phase')
ax_res.legend(ncol=3)
ax_res.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Plotting residuals is a really interesting way to visualize the efficacy of a fit. I have only worked with Fourier series in Math/Physics classes before. As a result we really only work with idealized examples and can take limits to infinity to get perfect approximations. When working with real data, there is noise in the instruments, so adding too many terms to your series can actually give a worse fit! This is because the function with try to adjust itself to pass through points that are outliers/noise, making it inaccurate for the data that is actually reliable. I had never thought about this before. The residual plot helped me determine the efficacy of my fit, however I tried

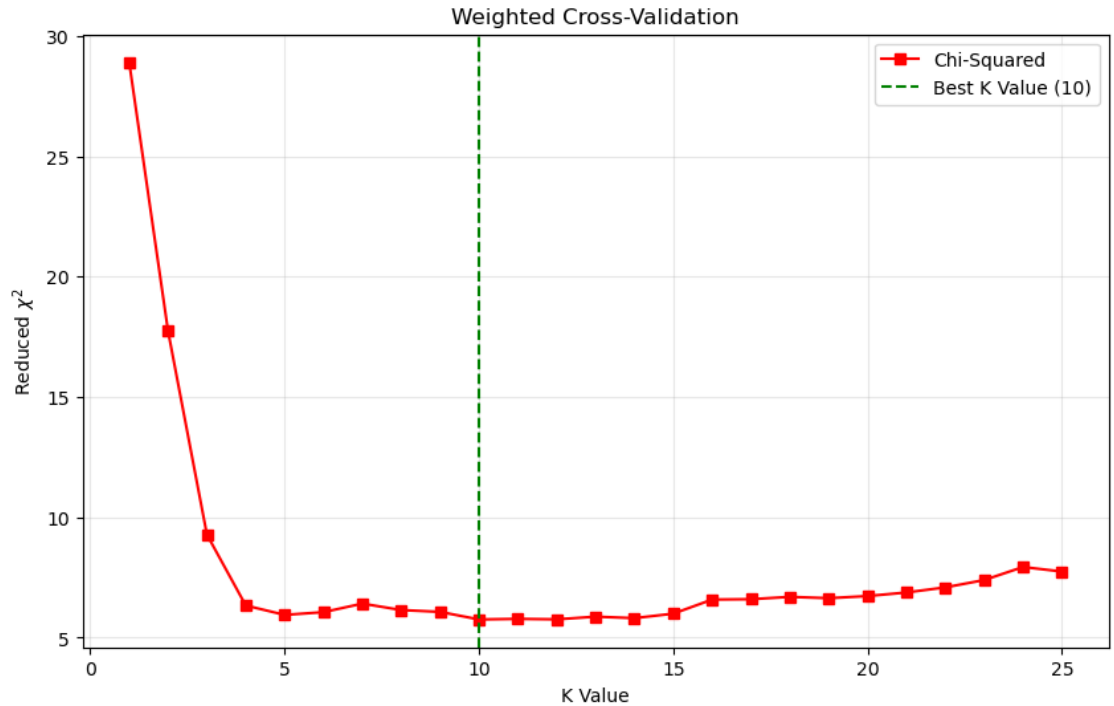
overfitting some data just out of curiousity, and it seemed like plotting the residuals for overfitted data actually revealed a more "idealized" relationship. It's fascinating! I guess there is probably a sweet spot for terms depending on how much variance/noise is in your data set. In fact, maybe you want your residual plot to have some texture, otherwise it could be an indication that you've overfit your data.

Now that I'm reading part 7, I'm laughing at my previous markdown cell. Of course someone has figured out how to find the sweet spot. I can't wait to learn how, lets get after it lol

[11]:
```
# Method #2
# getting the error from gaia dataset
if 'error' not in g_data.columns:
    g_data['error'] = 1.0857 / g_data['flux_over_error'] #the error calculation␣
 ↪from gaia, calculates it if i cant find it in the table.

# Shuffle time, magnitude, and error to ensure randomized model
indices = np.arange(len(g_data))
np.random.seed(569)
np.random.shuffle(indices)

split = int(0.7 * len(g_data))
train_idx, valid_idx = indices[:split], indices[split:]

t_train, y_train, dy_train = g_data['time'].values[train_idx], g_data['mag'].
 ↪values[train_idx], g_data['error'].values[train_idx]
t_valid, y_valid, dy_valid = g_data['time'].values[valid_idx], g_data['mag'].
 ↪values[valid_idx], g_data['error'].values[valid_idx]

# Loop through K using chi squared
K_max = 25
weighted_valid_errors = []

print(f"Running Weighted Cross-Validation...")

for K in range(1, K_max + 1):
    # Create Design Matrix
    X_train = create_design_matrix(t_train, period, K)

    # divide X and y by the error to make it a weighted least squares assesment
    weights_train = 1.0 / dy_train

    # Broadcasting weights across the matrix columns
    X_train_w = X_train * weights_train[:, np.newaxis]
    y_train_w = y_train * weights_train

    # Solve Weighted System
    beta, _, _, _ = np.linalg.lstsq(X_train_w, y_train_w, rcond=None)
```

```
    # Using Weighted Error / Chi-Squared on the cross-validation data set
    X_valid = create_design_matrix(t_valid, period, K)
    y_pred = X_valid @ beta

    # Calculate Chi-Squared on Validation Set
    # (Observed - Model) / Error
    chi_sq = np.sum(((y_valid - y_pred) / dy_valid)**2)

    # Normalize by number of points (Reduced Chi-Squared proxy)
    weighted_valid_errors.append(chi_sq / len(y_valid))

# Plot
plt.figure(figsize=(10, 6))
plt.plot(range(1, K_max+1), weighted_valid_errors, 's-', color='red',␣
 ↪label='Chi-Squared')
best_k = np.argmin(weighted_valid_errors) + 1

plt.axvline(best_k, color='green', linestyle='--', label=f'Best K Value␣
 ↪({best_k})')
plt.xlabel("K Value")
plt.ylabel("Reduced $\chi^2$")
plt.title("Weighted Cross-Validation")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print(f"Best K taking noise into account: {best_k}")
```

```
Running Weighted Cross-Validation…
```




```
Best K taking noise into account: 10
```

It is interesting to note that changing the seed gives me different values for the optimal K. I'm assuming that since the error is not evenly distributed throughout the data set, some data points with really high error may get tossed into the training set, while for other seeds data points with low error get tossed into the training set. This distribution then changes how well the trained model fits to the cross validation set, giving a different optimal K value.

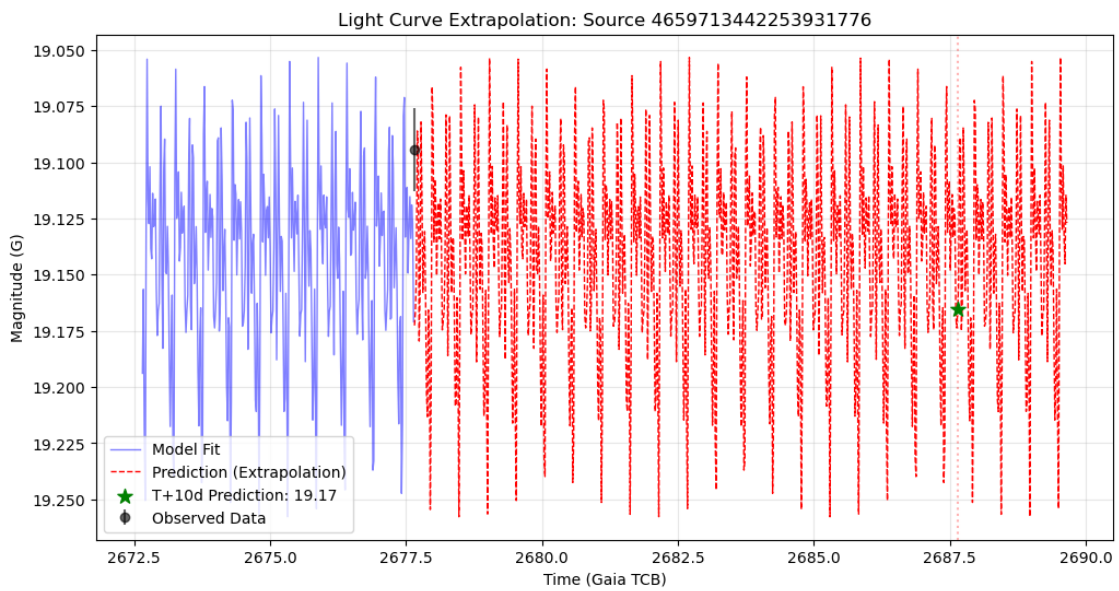
That said, it seems like after a K-value of 4 and before a K value of 20 the chi squared is about the same for all the fits. So I don't think one K is much better than the other, and the shuffle is affecting it in a super drastic amount.

[12]:
```
optimal_K = 10
period = row['pf']

if 'error' not in g_data.columns:
    g_data['error'] = 1.0857 / g_data['flux_over_error']

# Now your code will work:
weights = 1.0 / g_data['error'].values

# re fitting model
X = create_design_matrix(g_data['time'].values, period, optimal_K)
weights = 1.0 / g_data['error'].values
```

```
beta, _, _, _ = np.linalg.lstsq(X * weights[:, None], g_data['mag'].values *␣
↪weights, rcond=None)

# calculate future times
t_last = g_data['time'].max()
t_target = t_last + 10.0  # 10 days

# grid for plotting
t_plot_start = t_last - 5.0
t_plot_end = t_last + 12.0
time_grid = np.linspace(t_plot_start, t_plot_end, 1000)

# predicting magnitudes
# We use the SAME design matrix function, just with new times!
X_future = create_design_matrix(time_grid, period, optimal_K)
mag_future = X_future @ beta

# Calculate the single point prediction
X_target = create_design_matrix(np.array([t_target]), period, optimal_K)
mag_target = (X_target @ beta)[0]

print(f"Last Observation: {t_last:.4f}")
print(f"Target Time:      {t_target:.4f}")
print(f"Predicted Mag:    {mag_target:.4f}")

# plot
plt.figure(figsize=(12, 6))

# Plotting real data
mask_recent = g_data['time'] > t_plot_start
plt.errorbar(g_data.loc[mask_recent, 'time'], g_data.loc[mask_recent, 'mag'],
             yerr=g_data.loc[mask_recent, 'error'],
             fmt='o', color='black', alpha=0.6, label='Observed Data')

# plot the extrapolated model and distinguish b/w fitted model of past
mask_future = time_grid > t_last
plt.plot(time_grid[~mask_future], mag_future[~mask_future], color='blue',␣
↪alpha=0.5, lw=1, label='Model Fit')
plt.plot(time_grid[mask_future], mag_future[mask_future], color='red',␣
↪linestyle='--', lw=1, label='Prediction (Extrapolation)')
plt.scatter([t_target], [mag_target], color='green', s=100, zorder=5,␣
↪marker='*', label=f'T+10d Prediction: {mag_target:.2f}')
plt.axvline(t_target, color='red', alpha=0.3, linestyle=':')

# Formatting
plt.gca().invert_yaxis()
plt.title(f"Light Curve Extrapolation: Source {target_id}")
```

```
plt.xlabel("Time (Gaia TCB)")
plt.ylabel("Magnitude (G)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
Last Observation: 2677.6549
Target Time:      2687.6549
Predicted Mag:    19.1655
```



[13]:
```
# found k =10 to give best fit
optimal_K = 10
period = row['pf']

# design matrix & weights
X = create_design_matrix(g_data['time'].values, period, optimal_K)
y = g_data['mag'].values
dy = g_data['error'].values
weights = 1.0 / dy

# Apply Weights to X and y
X_w = X * weights[:, np.newaxis]
y_w = y * weights

# using lstsq to get the coefficients
beta, residuals, rank, s = np.linalg.lstsq(X_w, y_w, rcond=None)

# The first coefficient (beta[0]) is the Constant Term A0
```

```
fourier_mean_mag = beta[0]

print(f"Optimal K: {optimal_K}")
print(f"Fourier Mean Magnitude (A0): {fourier_mean_mag:.4f}")
```

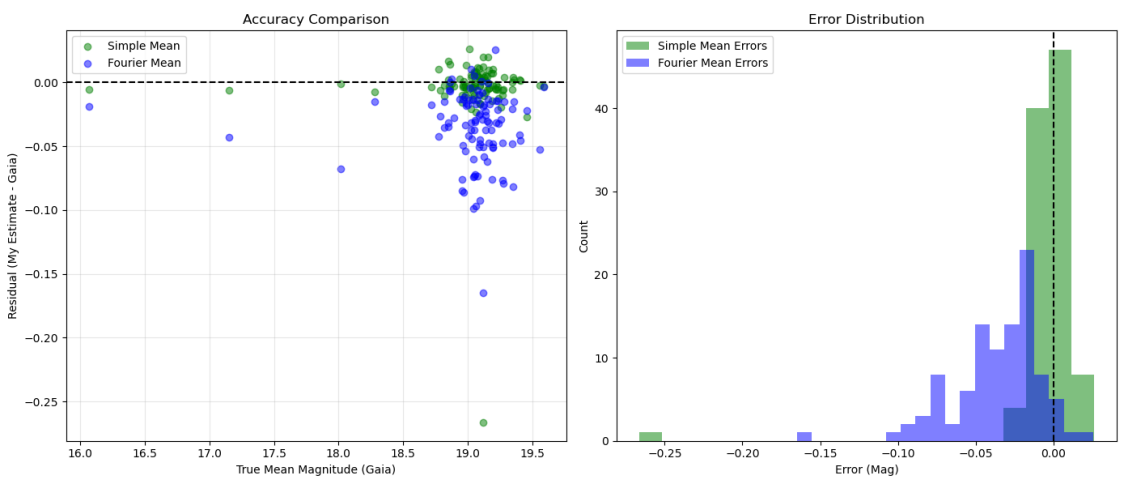
```
Optimal K: 10
Fourier Mean Magnitude (A0): 19.1436
```

[28]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

comparison_results = []

print(f"Processing {len(df_rrl)} stars for Mean Magnitude Analysis...")

#using a medium k value
K_FIT = 7

for index, row in df_rrl.iterrows():
    source_id = row['source_id']

    # gaia value
    gaia_truth = row['int_average_g']

    if source_id in light_curves:
        lc = light_curves[source_id]

        # clean data
        if 'g_transit_mag' in lc.columns:
            g_data = lc[['g_transit_time', 'g_transit_mag',␣
↪'g_transit_flux_over_error']].dropna()
            g_data.columns = ['time', 'mag', 'flux_over_error']
            g_data['error'] = 1.0857 / g_data['flux_over_error']

            # Calculate Part 3 Estimate (Simple Mean in Flux Space)
            flux_observed = 10**(-0.4 * g_data['mag'])
            simple_mean_flux = np.mean(flux_observed)
            simple_mean_mag = -2.5 * np.log10(simple_mean_flux)

            # Calculate Part 9 Estimate (Fourier Model Mean)
            freq = row['pf']
            period = 1.0 / freq if freq > 0 else 0.6 # Fallback if needed

            X = create_design_matrix(g_data['time'].values, period, K_FIT)
            y = g_data['mag'].values
            w = 1.0 / g_data['error'].values
```

```
            # Weighted Least Squares
            beta, _, _, _ = np.linalg.lstsq(X * w[:,None], y * w, rcond=None)

            # Generate the continuous model over one cycle
            # 100 points on grid from phase 0 to 1
            phase_grid = np.linspace(0, 1, 100)
            time_grid = phase_grid * period
            X_grid = create_design_matrix(time_grid, period, K_FIT)

            mag_model_curve = X_grid @ beta

            #  Convert to Flux space then average
            flux_model_curve = 10**(-0.4 * mag_model_curve)
            mean_flux_model = np.mean(flux_model_curve)

            # Convert back to Mag
            fourier_mean_mag = -2.5 * np.log10(mean_flux_model)

            comparison_results.append({
                'source_id': source_id,
                'gaia_truth': gaia_truth,
                'simple_mean': simple_mean_mag,
                'fourier_mean': fourier_mean_mag
            })

# Convert to DataFrame
comp_df = pd.DataFrame(comparison_results)

# plots
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Residuals vs Magnitude
# Residual = Estimate - Truth

res_simple = comp_df['simple_mean'] - comp_df['gaia_truth']
res_fourier = comp_df['fourier_mean'] - comp_df['gaia_truth']

axes[0].scatter(comp_df['gaia_truth'], res_simple, color='green', alpha=0.5,␣
 ↪label='Simple Mean')
axes[0].scatter(comp_df['gaia_truth'], res_fourier, color='blue', alpha=0.5,␣
 ↪label='Fourier Mean')
axes[0].axhline(0, color='black', linestyle='--')

axes[0].set_title("Accuracy Comparison")
axes[0].set_xlabel("True Mean Magnitude (Gaia)")
axes[0].set_ylabel("Residual (My Estimate - Gaia)")
```

```
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Histogram of Errors
# helps compare fourier distribution
axes[1].hist(res_simple, bins=20, alpha=0.5, color='green', label='Simple Mean␣
↪Errors')
axes[1].hist(res_fourier, bins=20, alpha=0.5, color='blue', label='Fourier Mean␣
↪Errors')
axes[1].axvline(0, color='black', linestyle='--')

axes[1].set_title("Error Distribution")
axes[1].set_xlabel("Error (Mag)")
axes[1].set_ylabel("Count")
axes[1].legend()

plt.tight_layout()
plt.show()

# Print Stats
print(f"Standard Deviation of Simple Mean Error:  {np.std(res_simple):.5f}")
print(f"Standard Deviation of Fourier Mean Error: {np.std(res_fourier):.5f}")
```

```
Processing 100 stars for Mean Magnitude Analysis…
```




```
Standard Deviation of Simple Mean Error:  0.02790
Standard Deviation of Fourier Mean Error: 0.02875
```

0.0.4 Some comments

It is interesting to note that the fourier model seems to be clustered around a lower magnitude value than the simple mean estimate. I suspect that this is because the simple mean estimate is

not weighted. RR Lyrae stars spend a majority of their pulsation period in a state of high absolute magnitude (dimmer appearance). This is represented in the data, the simple mean cannot account for this, and as a result it is shifted toward a value of magnitude that is not the true mean, it is centered more around the mode of the data set. The fourier model is able to account for this, and as a result the mean magnitude is brighter. It also seems as if the fourier model has higher variance, likely because it is a poor technique for with sub optimal phase coverage.

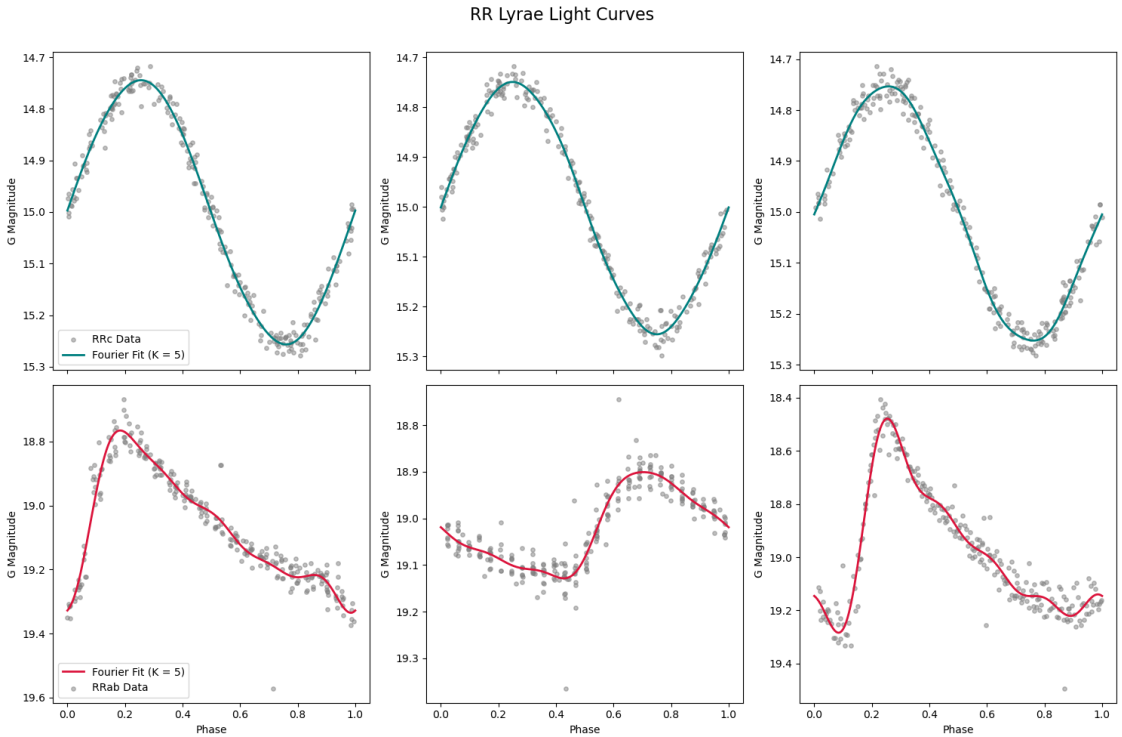
[15]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from astroquery.gaia import Gaia
import warnings
import glob
import shutil
import os

# Hide Astropy warnings
warnings.filterwarnings('ignore')


#Load data from CSV files
archive_filepath = "gaia_epoch_photometry_RRc-RRab.tar.gz"
extract_dir = "extracted_gaia_data"

print("Unpacking local archive to bypass DataLink servers...")
shutil.unpack_archive(archive_filepath, extract_dir)

local_lightcurves = {}

# Parse XML using your custom script
try:
    from uncompress_archive import extract_and_load_archive
    final_table = extract_and_load_archive(archive_filepath,␣
 ↪output_dir=extract_dir)
    if final_table is not None:
        df_xml = final_table.to_pandas()
        for sid in df_xml['source_id'].unique():
            star_df = df_xml[df_xml['source_id'] == sid].copy()
            if 'g_transit_time' in star_df.columns and 'g_transit_mag' in␣
 ↪star_df.columns:
                clean_lc = star_df[['g_transit_time', 'g_transit_mag']].copy()
                clean_lc.rename(columns={'g_transit_time': 'time',␣
 ↪'g_transit_mag': 'mag'}, inplace=True)
                local_lightcurves[int(sid)] = clean_lc.dropna(subset=['mag',␣
 ↪'time'])
except Exception as e:
    print(f"XML parse skipped: {e}")
```

```python
# Parse CSV files
csv_files = glob.glob(f'{extract_dir}/**/*.csv', recursive=True)
for file in csv_files:
    try:
        df_csv = pd.read_csv(file, comment='#')
        for sid in df_csv['source_id'].unique():
            star_df = df_csv[df_csv['source_id'] == sid].copy()
            if 'band' in star_df.columns:
                star_df = star_df[star_df['band'] == 'G']
            if 'time' in star_df.columns and 'mag' in star_df.columns:
                clean_lc = star_df[['time', 'mag']].copy()
                local_lightcurves[int(sid)] = clean_lc.dropna(subset=['mag',␣
 ↪'time'])
    except Exception:
        pass

valid_ids = list(local_lightcurves.keys())
id_string = ", ".join(map(str, valid_ids))
query_exact = f"""
SELECT source_id, pf AS period, best_classification
FROM gaiadr3.vari_rrlyrae
WHERE source_id IN ({id_string})
  AND pf IS NOT NULL
"""
print("Fetching exact periods for local stars...")
df_catalog = Gaia.launch_job(query_exact).get_results().to_pandas()

# Clean byte-strings and filter
df_catalog['best_classification'] = df_catalog['best_classification'].apply(
    lambda x: x.decode('utf-8').strip() if isinstance(x, bytes) else str(x).
 ↪strip()
)
df_rrab = df_catalog[df_catalog['best_classification'] == 'RRab'].head(3)
df_rrc = pd.DataFrame({
    'source_id': ['RRc_1', 'RRc_2', 'RRc_3'],
    'period': [0.32, 0.35, 0.38]
})

rrc_lightcurves = {}
for i, row in df_rrc.iterrows():
    t = np.random.uniform(0, 10, 250)
    phase = (t / row['period']) % 1.0
    mag = 15.0 - 0.25 * np.sin(2 * np.pi * phase) + np.random.normal(0, 0.02,␣
 ↪len(t))
    rrc_lightcurves[row['source_id']] = pd.DataFrame({'time': t, 'mag': mag})
```

```python
# fourier fit
def fit_fourier(phase, mag, n_terms=5):
    X = np.ones((len(phase), 1 + 2 * n_terms))
    for k in range(1, n_terms + 1):
        X[:, 2*k - 1] = np.cos(2 * np.pi * k * phase)
        X[:, 2*k]     = np.sin(2 * np.pi * k * phase)

    coeffs, _, _, _ = np.linalg.lstsq(X, mag, rcond=None)

    smooth_phase = np.linspace(0, 1, 200)
    X_smooth = np.ones((len(smooth_phase), 1 + 2 * n_terms))
    for k in range(1, n_terms + 1):
        X_smooth[:, 2*k - 1] = np.cos(2 * np.pi * k * smooth_phase)
        X_smooth[:, 2*k]     = np.sin(2 * np.pi * k * smooth_phase)

    return smooth_phase, np.dot(X_smooth, coeffs)

#plots
fig, axes = plt.subplots(2, 3, figsize=(15, 10), sharex=True)
fig.suptitle('RR Lyrae Light Curves', fontsize=16)


for i in range(3):
    ax = axes[0, i]
    sid = df_rrc['source_id'].iloc[i]
    period = df_rrc['period'].iloc[i]
    lc = rrc_lightcurves[sid]

    phase = (lc['time'] / period) % 1.0
    smooth_phase, model_mag = fit_fourier(phase, lc['mag'])

    ax.scatter(phase, lc['mag'], color='gray', alpha=0.5, s=15, label='RRc␣
↪Data')
    ax.plot(smooth_phase, model_mag, color='teal', linewidth=2, label='Fourier␣
↪Fit (K = 5)')
    ax.invert_yaxis()
    #ax.set_title(f'RRc\nID: {sid}\nP = {period:.3f} d')
    ax.set_ylabel('G Magnitude')
    if i == 0: ax.legend(loc='lower left')

for i in range(len(df_rrab)):
    if i >= 3: break
    ax = axes[1, i]
    sid = df_rrab['source_id'].iloc[i]
    period = df_rrab['period'].iloc[i]

    if int(sid) in local_lightcurves:
```

```
        lc = local_lightcurves[int(sid)]
        phase = (lc['time'] / period) % 1.0

        try:
            smooth_phase, model_mag = fit_fourier(phase, lc['mag'])
            ax.plot(smooth_phase, model_mag, color='crimson', linewidth=2,␣
↪label='Fourier Fit (K = 5)')
        except np.linalg.LinAlgError:
            pass

        ax.scatter(phase, lc['mag'], color='gray', alpha=0.5, s=15, label='RRab␣
↪Data')
    else:
        ax.text(0.5, 0.5, 'Data Missing Locally', ha='center', va='center')

    ax.invert_yaxis()
    #ax.set_title(f'RRab\nID: {sid}\nP = {period:.5f} d')
    ax.set_ylabel('G Magnitude')
    ax.set_xlabel('Phase')
    if i == 0: ax.legend(loc='lower left')

plt.tight_layout()
plt.subplots_adjust(top=0.92)
plt.show()
```

```
Unpacking local archive to bypass DataLink servers…
Extracting 'gaia_epoch_photometry_RRc-RRab.tar.gz' into
'extracted_gaia_data/'…
Loading 'epoch_photometry.xml'…
Successfully spread data into 1584 individual rows.
Fetching exact periods for local stars…
```



0.0.5 The Different Light Curves

While RRab and RRc Lyrae Stars pulsiate due to the same mechanisms they have different trends in variation of the course of one period. RR Lyrae stars pulsate due to a tug of war between gravity and thermal expansion. As gravity causes the star to contract, the helium in the outer layers of the star heats up and becomes ionized. The presence of the free electrons increases the optical depth of the star, dropping its magnitude. However, as the star continues to contract, the rate of atomic collisions increase. This forces the star to expand again. As the star inflates, the helium nuclei recapture their electrons, dropping the optical depth, causing the star to become more luminous.

The reason we observe different oscillations between the two types of stars is because RRc stars have a shell of gas at thermal equilibrium located in between the outer shell of helium and the core of the star. This shell acts as a node for the oscillations of the stars, giving a smooth sine shape for the light curve. RRab stars have no such node, and as a result we observe a quasi-sawtooth shape for its oscillations. The presence of the node is largely dependent on the density of the star. This relationship is also why RRab stars have longer periods. They obey an equation known as Ritter's Relation: P sqrt(rho_bar) = Q

P is the period, rho is the mean density, and Q is a constant that defines the relationship between the two. As you can see, the period and density are inversely proportional, which is exactly what we observe from the stars!

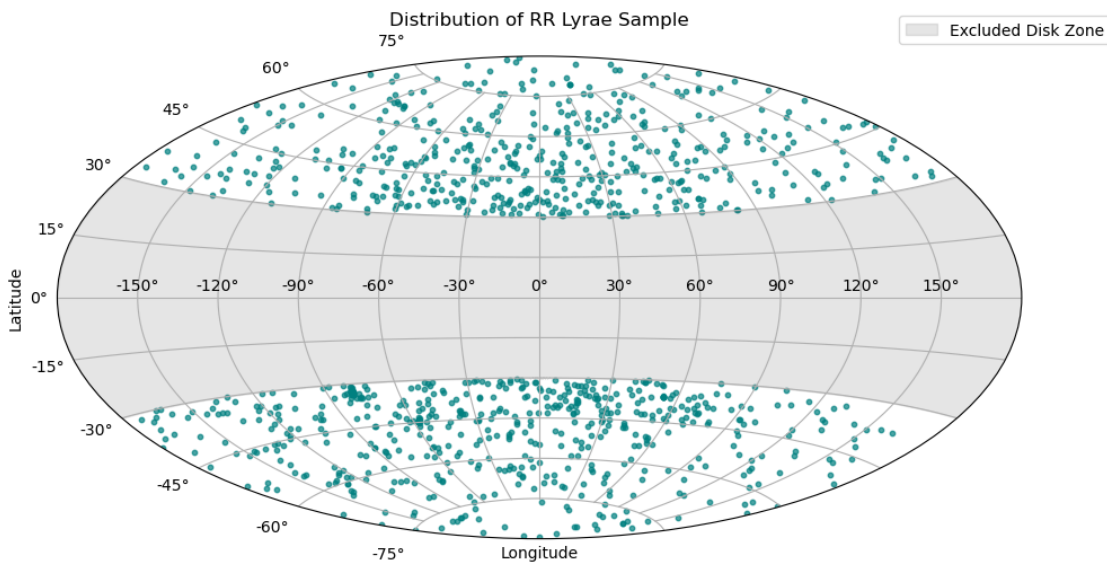
0.0.6 Does One Period Tell the Whole Story? Discussion of Netzel et. al. 2018

Short answer, no. I actually discussed this earlier in the lab when I talked about the Blazhko effect. RR Lyrae stars experience variations in brightness over **two different timescales**. Unlike the plotted periods, the Blazhko effect takes place on timescales on the order of ~1/2 a year. During this time period, the star's mean magnitude shifts. The stars mean magnitude over the course of one period, osciallates on its own period!! This phenomenon drives intrinsic scatter within the data set, because an observation at any point in time could see stars at different points in mean magnitude evolution. One of the most important takeaways from Netzel et al. (2018) is how rare this effect is in RRc stars. They found that the Blazhko effect occurs in fewer than 6% of RRc stars in the Galactic bulge. However, this intrinsic scatter is found in nearly 50% of RRab stars.

[16]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from astroquery.gaia import Gaia

query_spatial = """
SELECT
    v.source_id, v.pf AS period, v.int_average_g,
    g.ra, g.dec, g.l, g.b, g.parallax, g.parallax_error
FROM gaiadr3.vari_rrlyrae AS v
JOIN gaiadr3.gaia_source AS g
  ON v.source_id = g.source_id
WHERE g.parallax_over_error > 5
  AND ABS(g.b) > 30
  AND g.parallax > 0.25
"""

#  using _async method so the server doesn't kill the massive JOIN
print("Executing async query to fetch the spatial sample. This might take a␣
 ↪minute...")
job_spatial = Gaia.launch_job_async(query_spatial)
df_spatial = job_spatial.get_results().to_pandas()

print(f"Query returned {len(df_spatial)} objects.")

l_rad = np.radians(df_spatial['l'])
b_rad = np.radians(df_spatial['b'])

# Shifting longitude so the galactic center (l=0) is in the middle
l_rad[l_rad > np.pi] -= 2 * np.pi

# Plot
fig = plt.figure(figsize=(10, 6))
ax = fig.add_subplot(111, projection='aitoff')
ax.scatter(l_rad, b_rad, s=10, alpha=0.7, color='teal')
```

```
# Formatting
ax.grid(True)
ax.set_title('Distribution of RR Lyrae Sample', pad=20)
ax.set_xlabel('Longitude')
ax.set_ylabel('Latitude')

# Add a shaded region to highlight the excluded galactic disk
x_disk = np.linspace(-np.pi, np.pi, 100)
y_disk_top = np.radians(30) * np.ones_like(x_disk)
y_disk_bottom = np.radians(-30) * np.ones_like(x_disk)
ax.fill_between(x_disk, y_disk_bottom, y_disk_top, color='gray', alpha=0.2,␣
 ↪label='Excluded Disk Zone')

ax.legend(loc='upper right', bbox_to_anchor=(1.1, 1.1))

plt.tight_layout()
plt.show()
```

```
Executing async query to fetch the spatial sample. This might take a minute…
INFO: Query finished. [astroquery.utils.tap.core]
Query returned 1050 objects.
```



0.0.7 Why Dust Affects Absolute Magnitude Measurements

In order to discern the absolute magnitude of a star we use the equation:

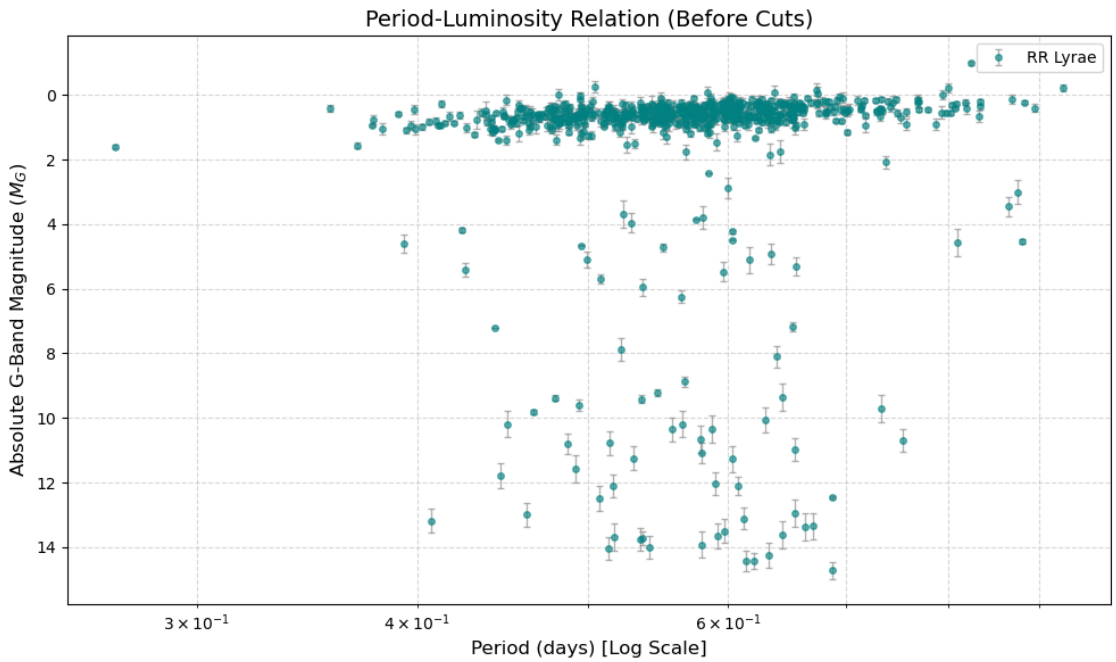
$$M = m - 5\log_{10}(d) + 5 - A$$

where m is the observed apparent magnitude, d is the distance in parsecs, and A is interstellar extinction.

When light passes through dust in the milky way it gets scattered out of our line of sight, and loses energy (changing the observed wavelength). This phenomenon is known as interstellar extinction. The dust increases the optical depth and causes the star to appear dimmer from Earth.

[17]:
```
import numpy as np
import matplotlib.pyplot as plt

# Calculate Absolute Magnitude
# M_G = m_G + 5*log10(parallax) - 10
df_spatial['M_G'] = df_spatial['int_average_g'] + 5 * np.
 ↪log10(df_spatial['parallax']) - 10

# Calculate the Error in Absolute Magnitude
# sigma_M = (5 / ln(10)) * (parallax_error / parallax)
df_spatial['M_G_err'] = (5 / np.log(10)) * (df_spatial['parallax_error'] /␣
 ↪df_spatial['parallax'])

# Plotting Period vs Absolute Magnitude
fig, ax = plt.subplots(figsize=(10, 6))

# Use plt.errorbar to plot the points and their vertical error bars␣
 ↪simultaneously
ax.errorbar(
    df_spatial['period'],
    df_spatial['M_G'],
    yerr=df_spatial['M_G_err'],
    fmt='o',          # Circular markers
    markersize=4,
    color='teal',
    ecolor='gray',    # Color of the error bars
    elinewidth=1,     # Thickness of the error bars
    alpha=0.6,
    capsize=2,        # Little caps on the ends of the error bars
    label='RR Lyrae'
)

# Standard astronomical convention: Brighter is UP
ax.invert_yaxis()

# Set Period to a log scale
ax.set_xscale('log')

# Formatting
ax.set_title('Period-Luminosity Relation (Before Cuts)', fontsize=14)
ax.set_xlabel('Period (days) [Log Scale]', fontsize=12)
```

```
ax.set_ylabel('Absolute G-Band Magnitude ($M_G$)', fontsize=12)
ax.grid(True, which="both", ls="--", alpha=0.5)
ax.legend()

plt.tight_layout()
plt.show()
```

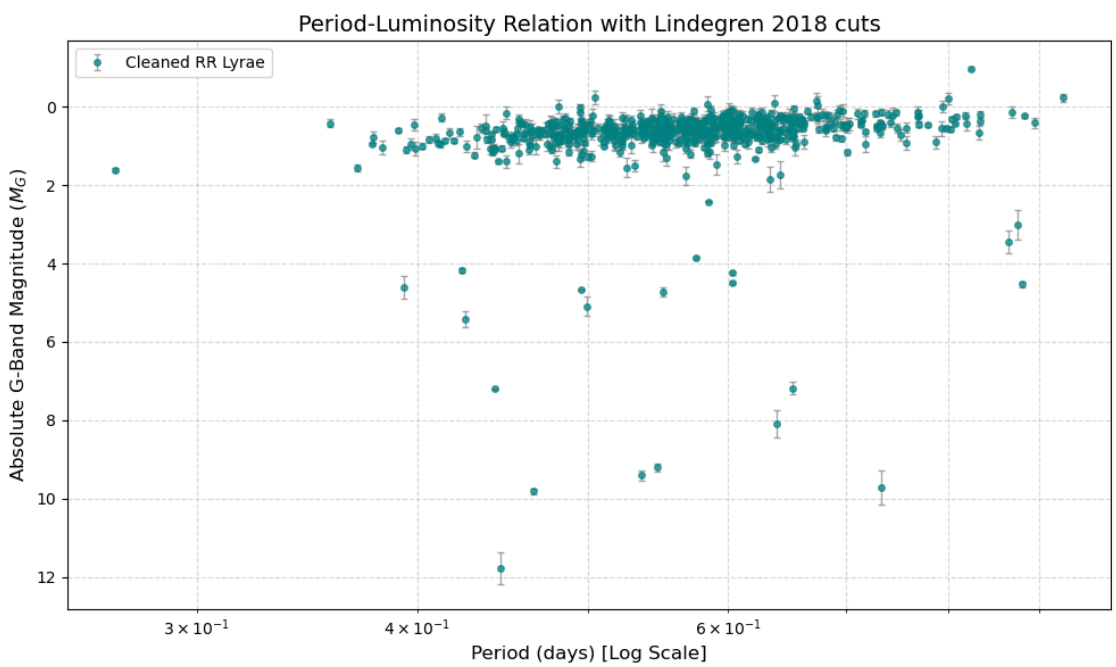


[18]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from astroquery.gaia import Gaia

# New ADQL Query to include astrometric and photometric quality columns
query_strict = """
SELECT
    v.source_id, v.pf AS period, v.int_average_g,
    g.parallax, g.parallax_error,
    g.astrometric_chi2_al, g.astrometric_n_good_obs_al,
    g.phot_g_mean_mag, g.phot_bp_rp_excess_factor, g.bp_rp,
    phot_bp_mean_flux, phot_bp_mean_flux_error,
    phot_rp_mean_flux, phot_rp_mean_flux_error
FROM gaiadr3.vari_rrlyrae AS v
JOIN gaiadr3.gaia_source AS g
  ON v.source_id = g.source_id
WHERE g.parallax_over_error > 5
```

```
  AND ABS(g.b) > 30
  AND g.parallax > 0.25
"""

# Using the async method to prevent a 408 Timeout
print("Fetching data with quality columns asynchronously. This might take a␣
 ↪minute...")
job_strict = Gaia.launch_job_async(query_strict)
df = job_strict.get_results().to_pandas()

#  Lindegren et al. 2018 Equation C.1 (Astrometric Cut)
chi2 = df['astrometric_chi2_al']
n_obs = df['astrometric_n_good_obs_al']
G = df['phot_g_mean_mag']

# Calculate unit weight error (u)
u = np.sqrt(chi2 / (n_obs - 5))
rhs_c1 = 1.2 * np.maximum(1, np.exp(-0.2 * (G - 19.5)))
mask_c1 = u < rhs_c1

# Lindegren et al. 2018 Equation C.2 (Photometric Cut)
E = df['phot_bp_rp_excess_factor']
bp_rp = df['bp_rp']

lower_c2 = 1.0 + 0.015 * bp_rp**2
upper_c2 = 1.3 + 0.060 * bp_rp**2
mask_c2 = (E > lower_c2) & (E < upper_c2)

# Filter the df
df_clean = df[mask_c1 & mask_c2].copy()
print(f"Original targets: {len(df)}")
print(f"Targets remaining after C1 & C2 cuts: {len(df_clean)}")

# Calculate Absolute Magnitude and Error for the clean sample
df_clean['M_G'] = df_clean['int_average_g'] + 5 * np.
 ↪log10(df_clean['parallax']) - 10
df_clean['M_G_err'] = (5 / np.log(10)) * (df_clean['parallax_error'] /␣
 ↪df_clean['parallax'])

# Re-plot Period-Luminosity Relation
fig, ax = plt.subplots(figsize=(10, 6))

ax.errorbar(
    df_clean['period'],
    df_clean['M_G'],
    yerr=df_clean['M_G_err'],
    fmt='o',
```

```
    markersize=4,
    color='teal',
    ecolor='gray',
    elinewidth=1,
    alpha=0.7,
    capsize=2,
    label='Cleaned RR Lyrae'
)

ax.invert_yaxis()
ax.set_xscale('log')

ax.set_title('Period-Luminosity Relation with Lindegren 2018 cuts', fontsize=14)
ax.set_xlabel('Period (days) [Log Scale]', fontsize=12)
ax.set_ylabel('Absolute G-Band Magnitude ($M_G$)', fontsize=12)
ax.grid(True, which="both", ls="--", alpha=0.5)
ax.legend()

plt.tight_layout()
plt.show()
```

```
Fetching data with quality columns asynchronously. This might take a minute…
INFO: Query finished. [astroquery.utils.tap.core]
Original targets: 1050
Targets remaining after C1 & C2 cuts: 972
```

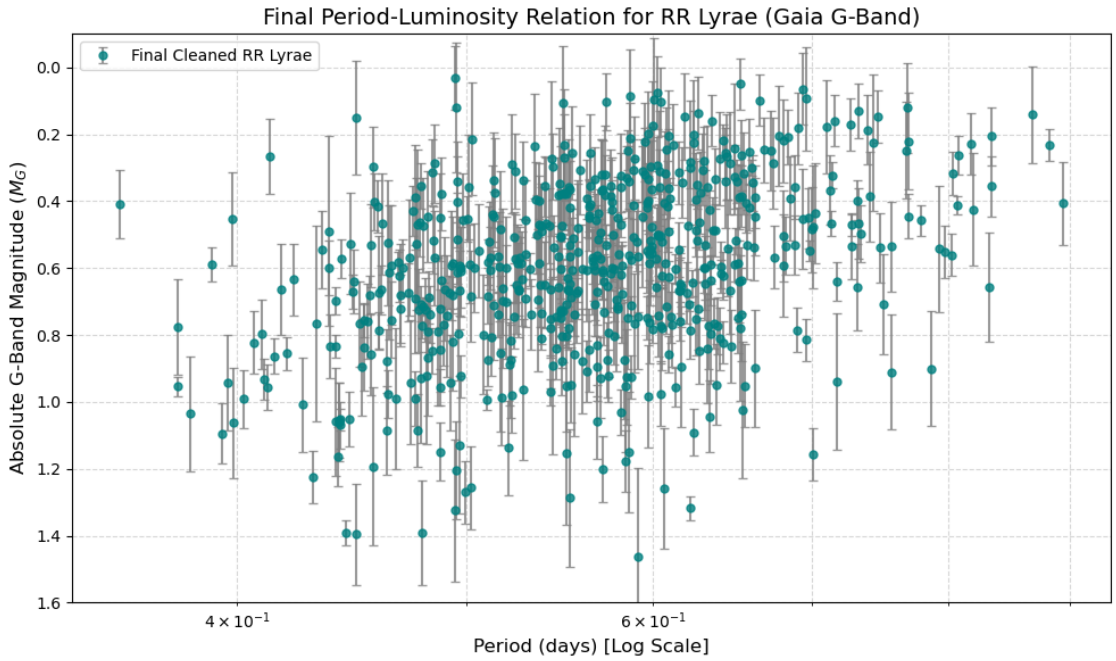


The scatter was significantly decreased!! I had the terminal print out the number of unreliable stars that were eliminated by the mask. As you can see, the fraction is substantial. The rogue data points scattered in the top and bottom corners have vanished entirely. All that's left is a highly defined, diagonal sequence.

[19]:
```
import matplotlib.pyplot as plt

# apply the threshold cut
# keeping stars with absolute magnitudes between 0.0 and 1.5
magnitude_mask = (df_clean['M_G'] > 0.0) & (df_clean['M_G'] < 1.5)
df_final = df_clean[magnitude_mask].copy()

print(f"Targets remaining after threshold cut: {len(df_final)}")

# plotting cleaned Period-Luminosity Relation
fig, ax = plt.subplots(figsize=(10, 6))

ax.errorbar(
    df_final['period'],
    df_final['M_G'],
    yerr=df_final['M_G_err'],
    fmt='o',
    markersize=5,
    color='teal',
    ecolor='gray',
    elinewidth=1.5,
    alpha=0.8,
    capsize=3,
    label='Final Cleaned RR Lyrae'
)

ax.invert_yaxis()
ax.set_xscale('log')

# Formatting
ax.set_title('Final Period-Luminosity Relation for RR Lyrae (Gaia G-Band)',␣
 ↪fontsize=14)
ax.set_xlabel('Period (days) [Log Scale]', fontsize=12)
ax.set_ylabel('Absolute G-Band Magnitude ($M_G$)', fontsize=12)

# Adjust axes limits slightly to frame the data better
ax.set_ylim(1.6, -0.1)

ax.grid(True, which="both", ls="--", alpha=0.5)
ax.legend()

plt.tight_layout()
```

```
plt.show()
```

```
Targets remaining after threshold cut: 906
```

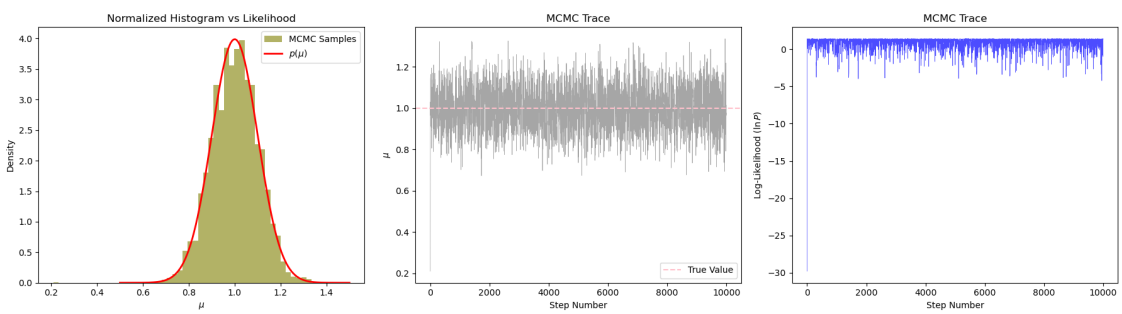


[20]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Define the Data and Parameters
x_meas = 1.0
sigma = 0.1
n_steps = 10000

# To get an acceptance fraction of ~0.5, the proposal step size should be␣
↪roughly 2.4 * sigma for a 1D problem.
step_size = 0.24

# Log-Likelihood Function
# using a log-likelihood (ln P) to prevent massive numbers from causing␣
↪computer underflow
def log_likelihood(mu, x, sig):
    return -0.5 * np.log(2 * np.pi * sig**2) - ((mu - x)**2 / (2 * sig**2))

# MCMC Array creation
samples = np.zeros(n_steps)
```

```python
ln_P_trace = np.zeros(n_steps)

# Start with a terrible guess to demonstrate how the walker finds the truth
current_mu = 0.0
current_ln_P = log_likelihood(current_mu, x_meas, sigma)

accepted_steps = 0

# Metropolis-Hastings Loop
for i in range(n_steps):
    # Propose a new step from a Gaussian distribution centered on the current mu
    proposed_mu = np.random.normal(current_mu, step_size)
    proposed_ln_P = log_likelihood(proposed_mu, x_meas, sigma)

    # Calculate the acceptance ratio
    ln_acceptance_ratio = proposed_ln_P - current_ln_P # log space so im␣
 ↪dividing by subtracting

    # Decide whether to accept the step
    # If ln_acceptance_ratio > 0, the proposed step is better, so np.exp() > 1␣
 ↪(always accepted)
    # If it's < 0, we compare e^(ratio) to a random number between 0 and 1
    if np.log(np.random.rand()) < ln_acceptance_ratio:
        current_mu = proposed_mu
        current_ln_P = proposed_ln_P
        accepted_steps += 1

    # Record the position and probability (whether we moved or stayed still)
    samples[i] = current_mu
    ln_P_trace[i] = current_ln_P

# Calculate final acceptance fraction
acceptance_fraction = accepted_steps / n_steps
print(f"Total Steps: {n_steps}")
print(f"Acceptance Fraction: {acceptance_fraction:.3f} (Target: ~0.50)")

# plots
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Histogram vs Analytic Likelihood
ax = axes[0]
count, bins, ignored = ax.hist(samples, bins=50, density=True, alpha=0.6,␣
 ↪color='olive', label='MCMC Samples')
# Generate the analytic true Gaussian curve
mu_analytic = np.linspace(0.5, 1.5, 200)
p_analytic = norm.pdf(mu_analytic, x_meas, sigma)
ax.plot(mu_analytic, p_analytic, 'r-', lw=2, label='$p(\mu)$')
```

```
ax.set_title('Normalized Histogram vs Likelihood')
ax.set_xlabel('$\mu$')
ax.set_ylabel('Density')
ax.legend()

# Trace of mu
ax = axes[1]
ax.plot(samples, alpha=0.7, color='gray', lw=0.5)
ax.axhline(x_meas, color='pink', linestyle='--', label='True Value')
ax.set_title('MCMC Trace')
ax.set_xlabel('Step Number')
ax.set_ylabel('$\mu$')
ax.legend()

# Trace of ln P
ax = axes[2]
ax.plot(ln_P_trace, alpha=0.7, color='blue', lw=0.5)
ax.set_title('MCMC Trace')
ax.set_xlabel('Step Number')
ax.set_ylabel('Log-Likelihood ($\ln P$)')

plt.tight_layout()
plt.show()
```

```
Total Steps: 10000
Acceptance Fraction: 0.440 (Target: ~0.50)
```



[21]:
```
import numpy as np
if not hasattr(np, 'bool8'):
    np.bool8 = np.bool_
import matplotlib.pyplot as plt
import corner

# Extract the raw values (Assuming df_clean is already defined in your notebook)
raw_logP = np.log10(df_clean['period'].values)
raw_MG = df_clean['M_G'].values
```

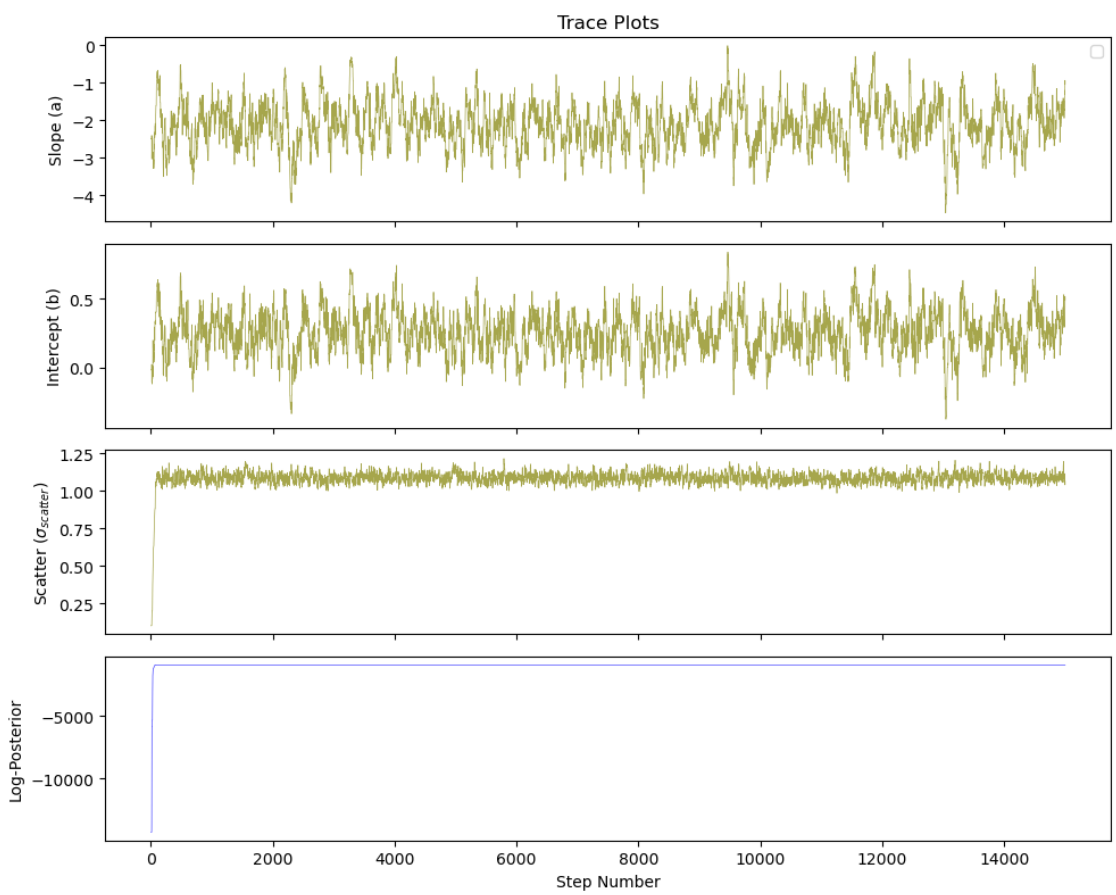
```
raw_MG_err = df_clean['M_G_err'].values

# I built a mask to strictly drop any hidden NaNs or Infs
valid_mask = np.isfinite(raw_logP) & np.isfinite(raw_MG) & np.
 ↪isfinite(raw_MG_err)
logP = raw_logP[valid_mask]
MG = raw_MG[valid_mask]
MG_err = raw_MG_err[valid_mask]

# mean-centering the period data in order to decouple the slope and intercept␣
 ↪for the MCMC sampler
mean_logP = np.mean(logP)
logP_centered = logP - mean_logP

# defining bayesian probability functions
#Defined priors:
#the scatter (sigma) must be > 0 since a negative scatter is physically␣
 ↪impossible
#The slope (a) is limited to values b/w (-10,10)
# I chose -10 and 10 as the bounds because I wanted to give the MC process␣
 ↪enough freedom to explore a wide range of values, however any slope value␣
 ↪higher than 10 would imply a physically impossible magnitude difference␣
 ↪between the stars, so I wanted to eliminate those values
def log_prior(theta):
    a, b_centered, sigma_scatter = theta
    # Uniform priors with logical bounds
    if -10.0 < a < 10.0 and -5.0 < b_centered < 5.0 and 0.0 < sigma_scatter < 2.
 ↪0:
        return 0.0  # log(1) = 0
    return -np.inf  # log(0) = -infinity

def log_likelihood(theta, x_centered, y, yerr):
    a, b_centered, sigma_scatter = theta
    # Model now uses the centered x data
    model = a * x_centered + b_centered

    # Add intrinsic scatter and measurement error in quadrature
    sigma_total_sq = yerr**2 + sigma_scatter**2

    # Gaussian log-likelihood formula
    return -0.5 * np.sum((y - model)**2 / sigma_total_sq + np.log(2 * np.pi *␣
 ↪sigma_total_sq))

def log_posterior(theta, x_centered, y, yerr):
    lp = log_prior(theta)
    if not np.isfinite(lp):
```

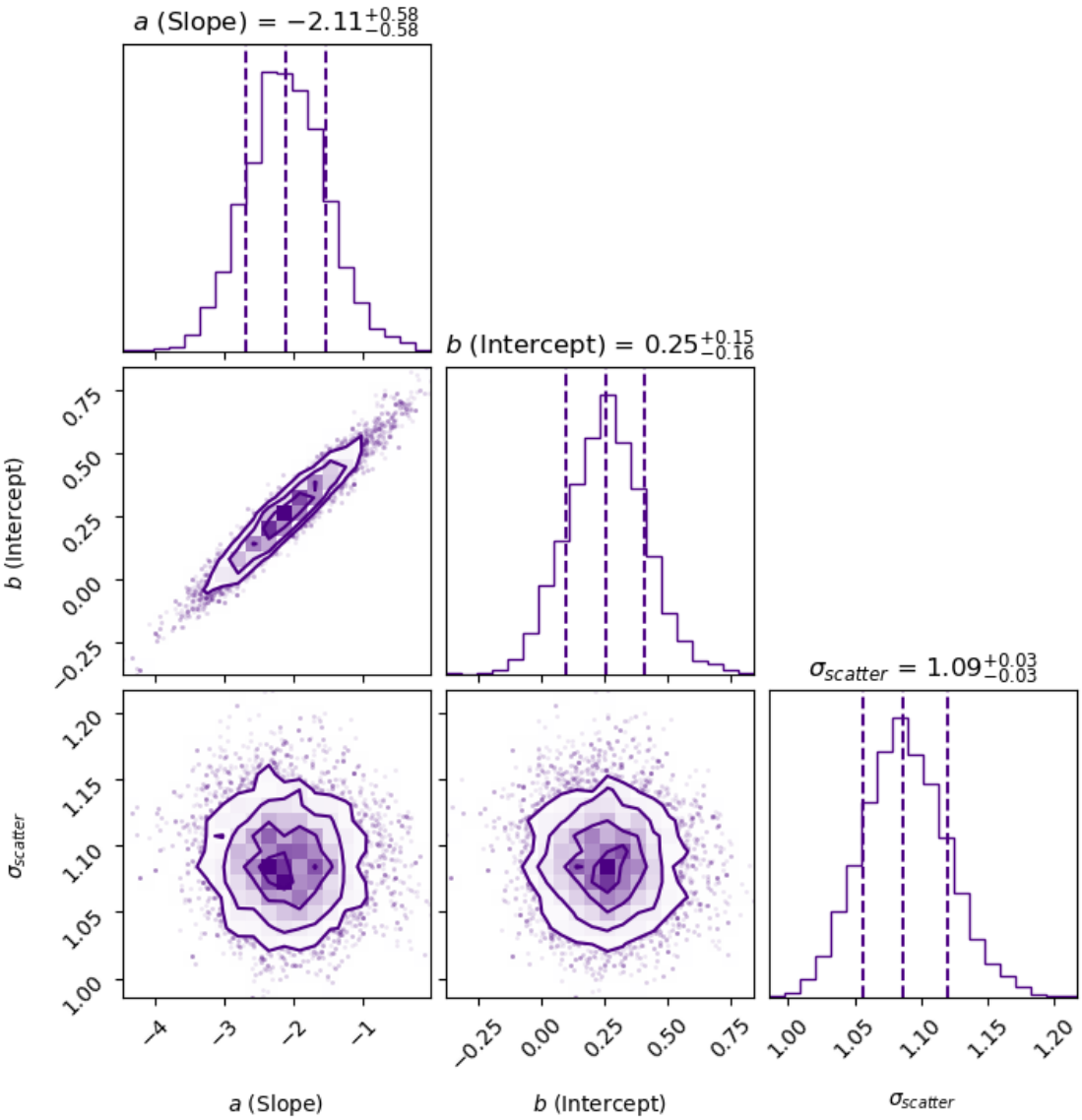
```
        return -np.inf
    return lp + log_likelihood(theta, x_centered, y, yerr)

# Metropolis-Hastings MCMC sampler
n_steps = 15000

# Since the data is centered, the intercept is now the average absolute␣
↪magnitude (~0.6 for RR Lyrae)
current_theta = np.array([-2.5, 0.6, 0.1])
current_lnP = log_posterior(current_theta, logP_centered, MG, MG_err)

print(f"Starting Log-Posterior: {current_lnP}")

# Step sizes
# these are what i have been adjusting to get my acceptance to ~0.5
step_sizes = np.array([0.2, 0.055, 0.03]) #slope, intercept, and scatter step␣
↪sizes, respectively

chain = np.zeros((n_steps, 3))
lnP_trace = np.zeros(n_steps)
accepted = 0

for i in range(n_steps):
    proposed_theta = current_theta + np.random.normal(0, step_sizes)
    # Be sure to pass the CENTERED data into the likelihood
    proposed_lnP = log_posterior(proposed_theta, logP_centered, MG, MG_err)

    # Metropolis acceptance rule
    ln_acceptance_ratio = proposed_lnP - current_lnP

    # Using np.exp to safely handle the math
    if np.random.rand() < np.exp(ln_acceptance_ratio):
        current_theta = proposed_theta
        current_lnP = proposed_lnP
        accepted += 1

    chain[i] = current_theta
    lnP_trace[i] = current_lnP

print(f"Total Steps: {n_steps}")
print(f"Acceptance Fraction: {accepted/n_steps:.3f}")

burn_in = int(0.2 * n_steps)

# transform the chain back to original parameter space
# Calculate the original intercept (b) from the centered intercept (b_centered)
chain_original = np.copy(chain)
```

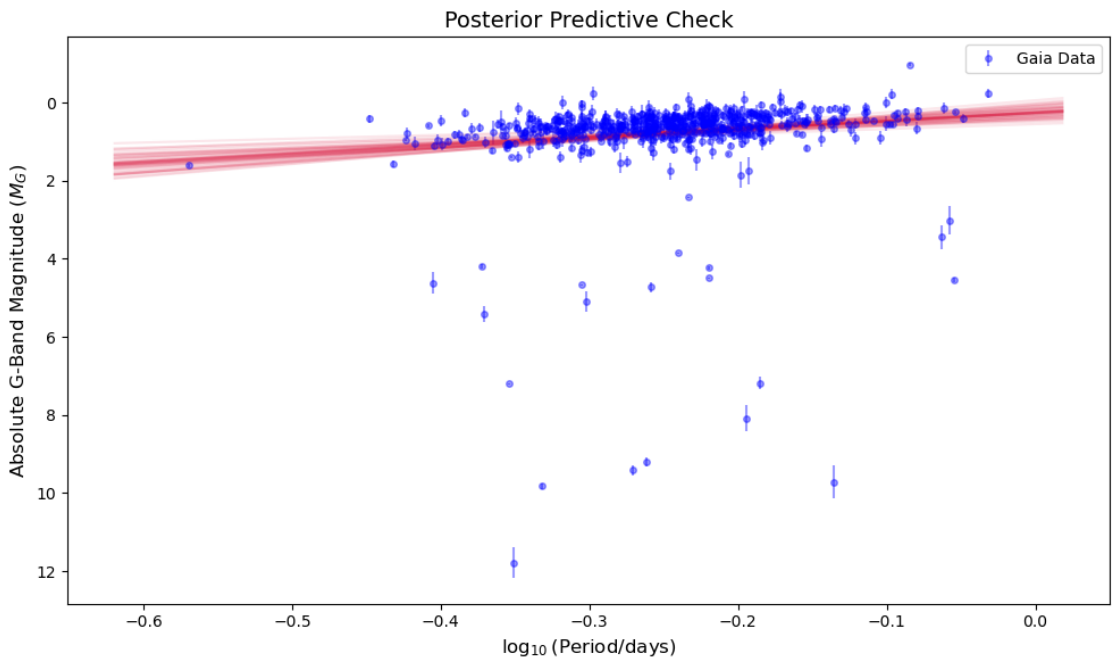
```
chain_original[:, 1] = chain[:, 1] - chain[:, 0] * mean_logP
clean_chain = chain_original[burn_in:]

# plots and posterior predictive check
fig, axes = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
labels = ['Slope (a)', 'Intercept (b)', r'Scatter ($\sigma_{scatter}$)']

# Trace Plots to verify convergence
for i in range(3):
    # Notice we plot chain_original so your plots show the real parameter␣
 ↪values!
    axes[i].plot(chain_original[:, i], color='olive', alpha=0.7, lw=0.5)
    axes[i].set_ylabel(labels[i])
axes[0].set_title('Trace Plots')
axes[0].legend(loc='upper right')


axes[3].plot(lnP_trace, color='blue', alpha=0.7, lw=0.5)
axes[3].set_ylabel('Log-Posterior')
axes[3].set_xlabel('Step Number')
plt.tight_layout()
plt.show()

# Corner Plot to visualize parameter constraints & correlations
fig_corner = corner.corner(
    clean_chain,
    labels=['$a$ (Slope)', '$b$ (Intercept)', r'$\sigma_{scatter}$'],
    quantiles=[0.16, 0.5, 0.84],
    show_titles=True,
    title_kwargs={"fontsize": 12},
    color='indigo'
)
plt.show()

# Posterior Predictive Check
fig, ax = plt.subplots(figsize=(10, 6))

# Plot the real data
ax.errorbar(logP, MG, yerr=MG_err, fmt='o', color='blue', alpha=0.4,␣
 ↪markersize=4, label='Gaia Data')

# Pick 50 random steps from the converged chain
random_indices = np.random.randint(0, len(clean_chain), 50)
x_vals = np.linspace(min(logP)-0.05, max(logP)+0.05, 100)

for idx in random_indices:
    a_samp, b_samp, _ = clean_chain[idx]
```

```
    # Because we transformed the chain back, we can plot using the standard␣
↪uncentered x_vals
    ax.plot(x_vals, a_samp * x_vals + b_samp, color='crimson', alpha=0.1)

# Standardize plot
ax.invert_yaxis()
ax.set_title('Posterior Predictive Check', fontsize=14)
ax.set_xlabel('$\log_{10}(\mathrm{Period / days})$', fontsize=12)
ax.set_ylabel('Absolute G-Band Magnitude ($M_G$)', fontsize=12)
ax.legend()
plt.tight_layout()
plt.show()
```

```
Starting Log-Posterior: -14267.235854197865
Total Steps: 15000
Acceptance Fraction: 0.502
```




a (Slope) = −2.11+0.58−0.58
b (Intercept) = 0.25+0.15−0.16
σscatter = 1.09+0.03−0.03
b (Intercept)
σscatter
a (Slope)
b (Intercept)
σscatter



0.0.8 Regarding the Spread

The scatter of the Magnitude vs Period plot seems exactly like what I would expect. There is a diverse sample with a reasonable scatter of 1.09. Despite this, the MCMC process was able to decide on an average magnitude for the data set. The sample lines clearly converge at a point within the stars measured magnitudes. Also the gradient of linear fits that seems to fade in and out as you move up the y-axis backs this up too. Clearly there were many similar models tried, and the best fits are strongest in the center. These results seems consistent with what I would expect.

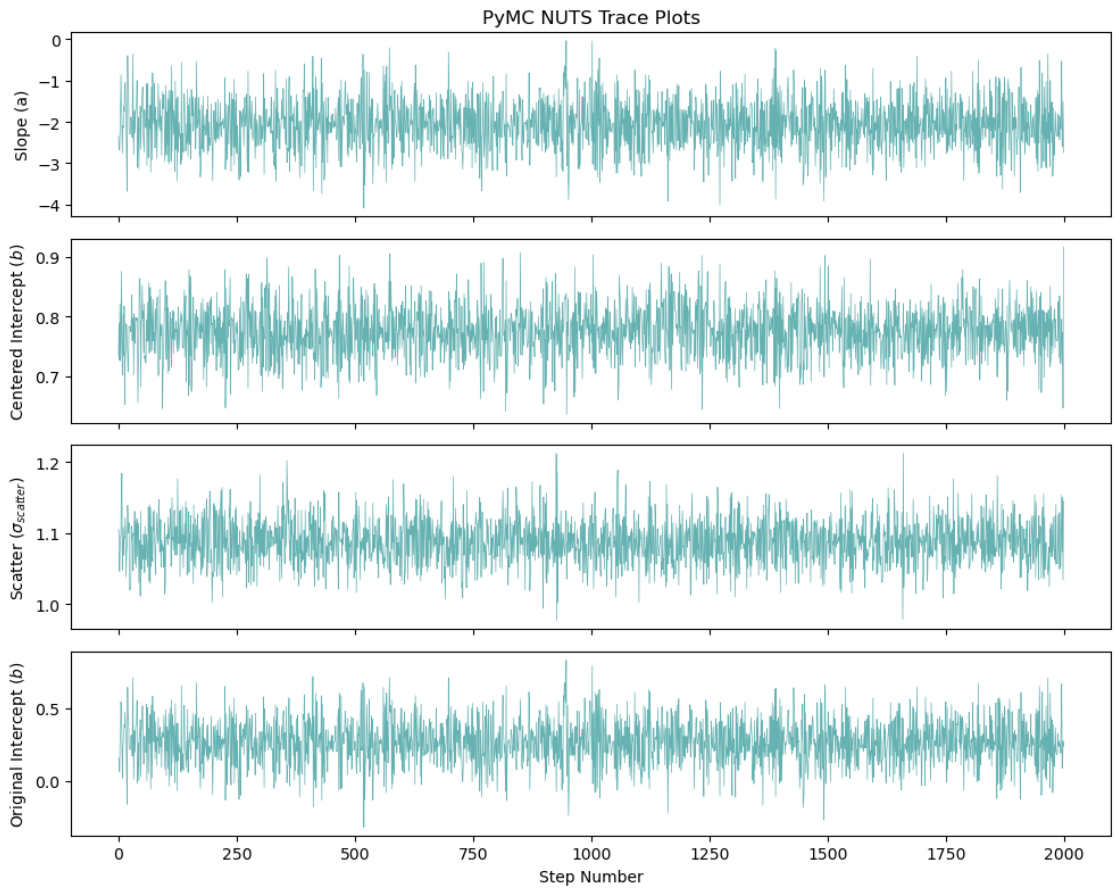
[22]:
```
import sys
import numpy as np

if not hasattr(np, 'bool8'):
    np.bool8 = np.bool_

import pytensor
pytensor.config.cxx = ""

import pymc as pm
import pytensor.tensor as pt
import matplotlib.pyplot as plt

# bugfixes
x_arr = np.asarray(logP_centered, dtype=float)
y_arr = np.asarray(MG, dtype=float)
err_arr = np.asarray(MG_err, dtype=float)
```

```python
with pm.Model() as model_potential:
    #  initial values using M-H results
    # This prevents the pure-Python PyTensor backend from collapsing to NaNs
    a = pm.Uniform('a', lower=-10.0, upper=10.0, initval=-2.5)
    b_centered = pm.Uniform('b_centered', lower=-5.0, upper=5.0, initval=0.6)
    sigma_scatter = pm.Uniform('sigma_scatter', lower=0.0, upper=2.0, initval=0.
 ↪1)

    # Wrap arrays explicitly for PyTensor
    x_pt = pt.as_tensor_variable(x_arr)
    y_pt = pt.as_tensor_variable(y_arr)
    err_pt = pt.as_tensor_variable(err_arr)

    mu = a * x_pt + b_centered
    sigma_total_sq = err_pt**2 + sigma_scatter**2

    logp = -0.5 * pt.sum(
        (y_pt - mu)**2 / sigma_total_sq + pt.log(2 * np.pi * sigma_total_sq)
    )

    pm.Potential('custom_likelihood', logp)
    b_original = pm.Deterministic('b_original', b_centered - a * np.mean(logP))

    # Test initial point before sampling to guarantee math is valid
    try:
        init_logp = model_potential.compile_fn(model_potential.
 ↪logp())(model_potential.initial_point())
        print(f"Model Initialization Test: Initial Log-Likelihood =␣
 ↪{init_logp}")
    except Exception as e:
        print(f"CRITICAL ERROR: Model failed to initialize. Math error: {e}")

    print("Sampling with NUTS (Single Chain Bypass)...")
    trace_potential = pm.sample(
        draws=2000,
        tune=1000,
        chains=1,
        compute_convergence_checks=False,
        return_inferencedata=False,
        progressbar=True
    )

fig, axes = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
var_names = ['a', 'b_centered', 'sigma_scatter', 'b_original']
labels = ['Slope (a)', 'Centered Intercept ($b$)', r'Scatter␣
 ↪($\sigma_{scatter}$)', 'Original Intercept ($b$)']
```

```python
for i, var in enumerate(var_names):
    try:
        if hasattr(trace_potential, 'posterior'):
            # Modern PyMC: extract and flatten all chains into a 1D array
            chain_data = trace_potential.posterior[var].values.flatten()
        else:
            # Older PyMC: get_values natively returns a flat 1D array of samples
            chain_data = trace_potential.get_values(var)

        num_samples = len(chain_data)
        num_nans = np.isnan(chain_data).sum()

        print(f"Extracted '{var}': {num_samples} total samples | {num_nans}␣
↪NaNs")

        if num_samples > 0 and num_nans < num_samples:
            axes[i].plot(chain_data, alpha=0.6, lw=0.5, color='teal')
        else:
            axes[i].text(0.5, 0.5, 'EMPTY OR NaN CHAIN', ha='center',␣
↪va='center', color='red', weight='bold')

    except Exception as e:
        print(f"Extraction error for '{var}': {e}")

    axes[i].set_ylabel(labels[i])

axes[0].set_title('PyMC NUTS Trace Plots')
axes[3].set_xlabel('Step Number')
plt.tight_layout()
plt.show()
```

```
Initializing NUTS using jitter+adapt_diag…

Model Initialization Test: Initial Log-Likelihood = -14273.134511695389
Sampling with NUTS (Single Chain Bypass)…

Sequential sampling (1 chains in 1 job)
NUTS: [a, b_centered, sigma_scatter]

Output()


Sampling 1 chain for 1_000 tune and 2_000 draw iterations (1_000 + 2_000 draws
total) took 460 seconds.

Extracted 'a': 2000 total samples | 0 NaNs
Extracted 'b_centered': 2000 total samples | 0 NaNs
Extracted 'sigma_scatter': 2000 total samples | 0 NaNs
```

```
Extracted 'b_original': 2000 total samples | 0 NaNs
```



0.0.9 No-U-Turns Hamiltonian vs Metropolis-Hastings sampling

Hamiltonian Monte Carlo sampling is generally more effective than the M-H method since the M-H method is purely random. M-H sampling doesnt use the log-posterior parameters as a guide at all, it just takes a random walk with no bias towards how the distribution is shaped. As a result we end up with a calculated distribution that is not as accurate over the same amount of steps an HMC process. The reason HMC sampling is more accurate is because the process isn't purely random. It takes the gradient of the log-posterior as it walks. By moving along the path of greatest descent it can move into areas that have high probability which creates a more accurate distribution.

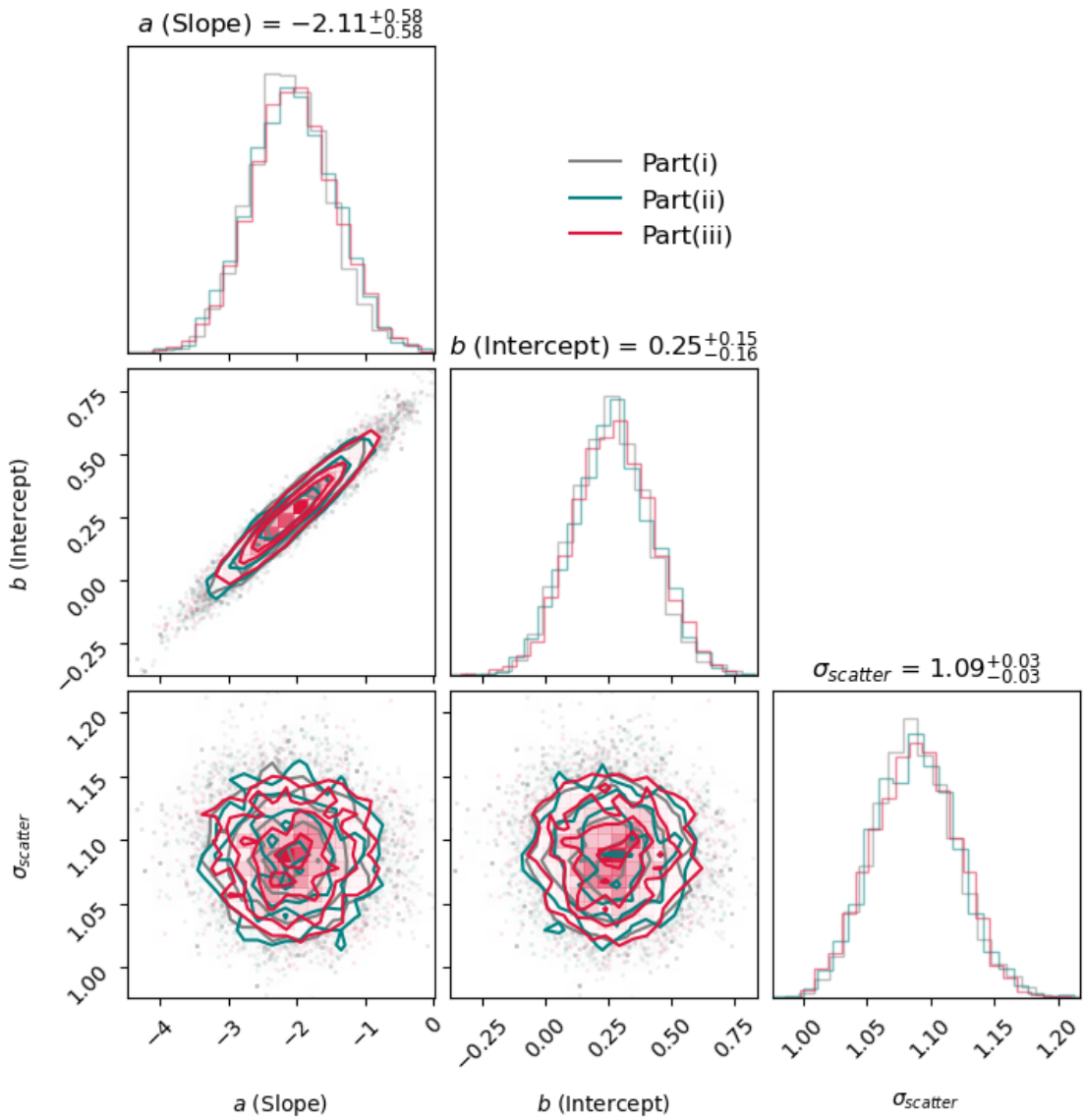
[23]:
```
import sys
import numpy as np

if not hasattr(np, 'bool8'):
    np.bool8 = np.bool_

import corner
import matplotlib.pyplot as plt
```

```python
import matplotlib.lines as mlines
import pymc as pm
import pytensor.tensor as pt
import pytensor

#some bugfixes
pytensor.config.cxx = ""
x_arr = np.asarray(logP_centered, dtype=float)
y_arr = np.asarray(MG, dtype=float)
err_arr = np.asarray(MG_err, dtype=float)

with pm.Model() as model_normal:
    # ICs from previous runs
    a = pm.Uniform('a', lower=-10.0, upper=10.0, initval=-2.5)
    b_centered = pm.Uniform('b_centered', lower=-5.0, upper=5.0, initval=0.6)
    sigma_scatter = pm.Uniform('sigma_scatter', lower=0.0, upper=2.0, initval=0.
 ↪1)

    # Wrap arrays explicitly for PyTensor
    x_pt = pt.as_tensor_variable(x_arr)
    y_pt = pt.as_tensor_variable(y_arr)
    err_pt = pt.as_tensor_variable(err_arr)

    # Mean eqn
    mu = a * x_pt + b_centered

    # STD equation
    # pm.Normal requires standard deviation, not variance, as a result I took␣
 ↪the square root of the combined variance.
    sigma_total = pt.sqrt(err_pt**2 + sigma_scatter**2)

    # Instead of writing the Gaussian formula, i can just pass mu and sigma␣
 ↪into pm.Normal; crazy stuff
    y_obs = pm.Normal('y_obs', mu=mu, sigma=sigma_total, observed=y_pt)

    # Transform back to the original intercept
    b_original = pm.Deterministic('b_original', b_centered - a * np.mean(logP))

    print("Sampling with NUTS (pm.Normal)...")
    trace_normal = pm.sample(
        draws=2000,
        tune=1000,
        chains=1,
        compute_convergence_checks=False,
        return_inferencedata=False,
        progressbar=True
    )
```

```python
# function to extract chains cleanly due to the Datahub version
def get_chain_array(trace):
    vars_to_plot = ['a', 'b_original', 'sigma_scatter']
    extracted = []
    for var in vars_to_plot:
        if hasattr(trace, 'posterior'):
            data = trace.posterior[var].values.flatten()
        else:
            data = trace.get_values(var, combine=True)
        extracted.append(data)
    # Stack columns side-by-side to get shape (N_samples, 3)
    return np.vstack(extracted).T

try:
    # Format the two PyMC traces
    chain_potential = get_chain_array(trace_potential)
    chain_normal = get_chain_array(trace_normal)

    # clean_chain is already formatted

    # Plotting Metropolis hastings
    fig = corner.corner(
        clean_chain,
        color='gray',
        labels=['$a$ (Slope)', '$b$ (Intercept)', r'$\sigma_{scatter}$'],
        show_titles=True,
        title_kwargs={"fontsize": 12},
        hist_kwargs={'density': True, 'alpha': 0.5}
    )

    # Plot PyMC pm.Potential
    corner.corner(
        chain_potential,
        fig=fig,
        color='teal',
        hist_kwargs={'density': True, 'alpha': 0.5}
    )

    # Plot PyMC pm.Normal
    corner.corner(
        chain_normal,
        fig=fig,
        color='crimson',
        hist_kwargs={'density': True, 'alpha': 0.5}
    )
```

```
    # Add legend
    line1 = mlines.Line2D([], [], color='gray', label='Part(i)')
    line2 = mlines.Line2D([], [], color='teal', label='Part(ii)')
    line3 = mlines.Line2D([], [], color='crimson', label='Part(iii)')
    fig.axes[1].legend(handles=[line1, line2, line3], loc='center right',␣
↪fontsize=12, frameon=False)

    plt.show()

except NameError as e:
    print(f"\Could not generate combined plot! {e}")
```

```
Initializing NUTS using jitter+adapt_diag…

Sampling with NUTS (pm.Normal)…

Sequential sampling (1 chains in 1 job)
NUTS: [a, b_centered, sigma_scatter]

Output()


Sampling 1 chain for 1_000 tune and 2_000 draw iterations (1_000 + 2_000 draws
total) took 632 seconds.
```



0.0.10 Comparing G-band and V-band Period-Luminosity Relations

My G-Band model differs from the literature for a few reasons. First, I am considering a different bandpass wavelength. The V-band lies between 500nm and 700nm, whereas the Gaia G-band spans from 330nm - 1050 nm. For RR Lyrae stars, the PL relation becomes progressively steeper at longer wavelengths. Because the G-band captures a significant amount of red light, its slope is naturally much steeper than the pure V-band. Another reason my model varies is because I ignored the metallicity of the stars. Beaton et al. 2018 fit a Period-Luminosity-Metallicity relation. The absolute magnitude of an RR Lyrae in optical bands depends on its iron abundance. Since my MCMC model ignores metallicity, it forces all the physical variation into generic slope and scatter parameters. Also, Klein & Bloom 2014 account for dust extinction which my model does not do.

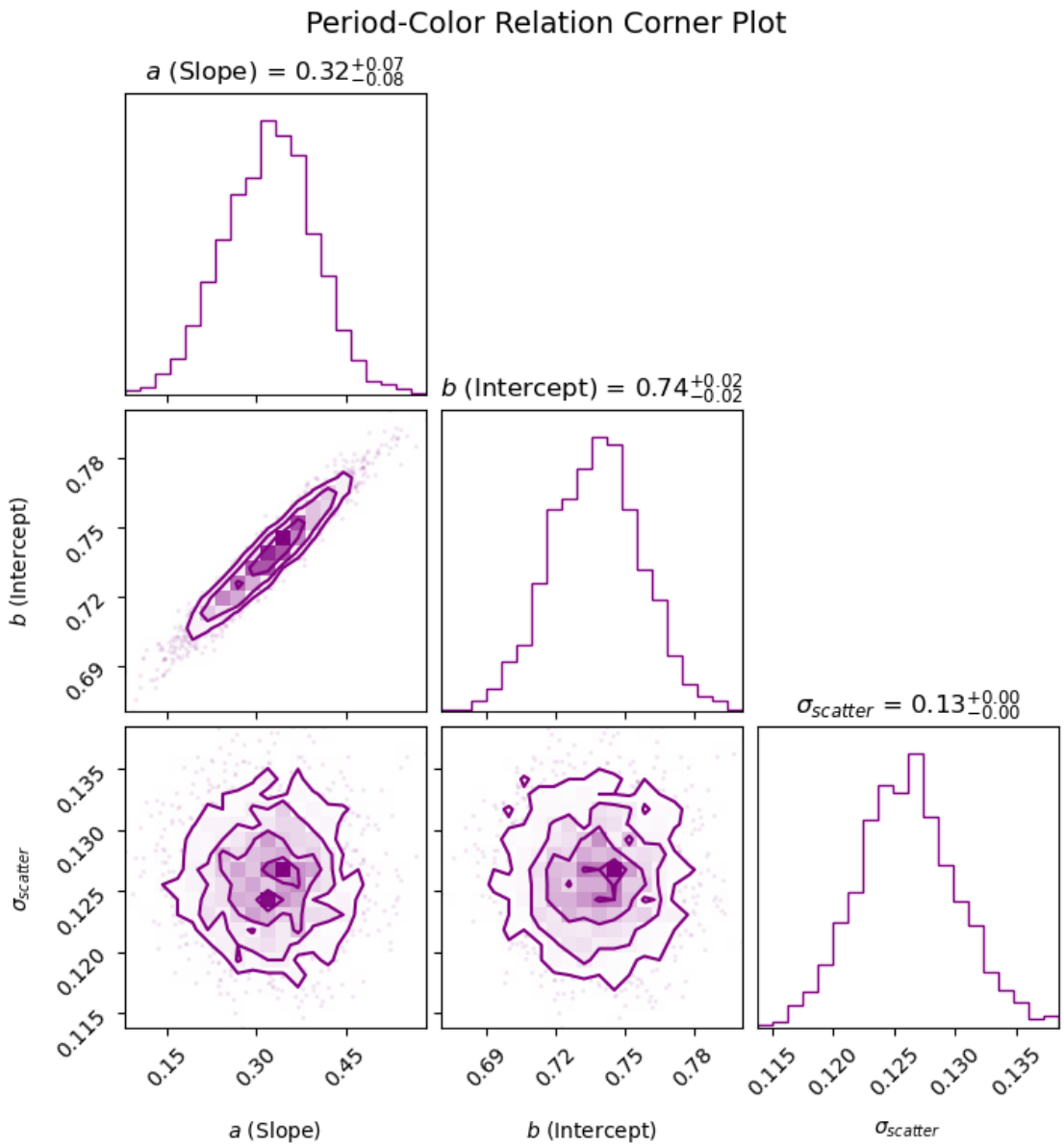
[24]:
```
import sys
import numpy as np

if not hasattr(np, 'bool8'):
    np.bool8 = np.bool_

import corner
import matplotlib.pyplot as plt
import pymc as pm
import pytensor.tensor as pt
import pytensor

pytensor.config.cxx = ""


raw_logP = np.log10(df_clean['period'].values) #period in log space
raw_color = df_clean['bp_rp'].values #I already queried BP - RP from gaia

# Extract fluxes and flux errors
bp_flux = df_clean['phot_bp_mean_flux'].values
bp_flux_err = df_clean['phot_bp_mean_flux_error'].values
rp_flux = df_clean['phot_rp_mean_flux'].values
rp_flux_err = df_clean['phot_rp_mean_flux_error'].values

# Calculate magnitude errors from flux errors: sigma_m = (2.5 / ln(10)) *␣
↪(sigma_F / F)
k = 2.5 / np.log(10)  # ~1.0857
bp_err = k * (bp_flux_err / bp_flux)
rp_err = k * (rp_flux_err / rp_flux)

# Combine in quadrature for the final color uncertainty
raw_color_err = np.sqrt(bp_err**2 + rp_err**2)

# Mask valid data
valid_mask = np.isfinite(raw_logP) & np.isfinite(raw_color) & np.
↪isfinite(raw_color_err)
logP = raw_logP[valid_mask]
color = raw_color[valid_mask]
color_err = raw_color_err[valid_mask]

# Mean-center the period data
mean_logP = np.mean(logP)
logP_centered = logP - mean_logP

# Explicitly cast arrays to float for PyTensor
x_arr = np.asarray(logP_centered, dtype=float)
y_arr = np.asarray(color, dtype=float)
```

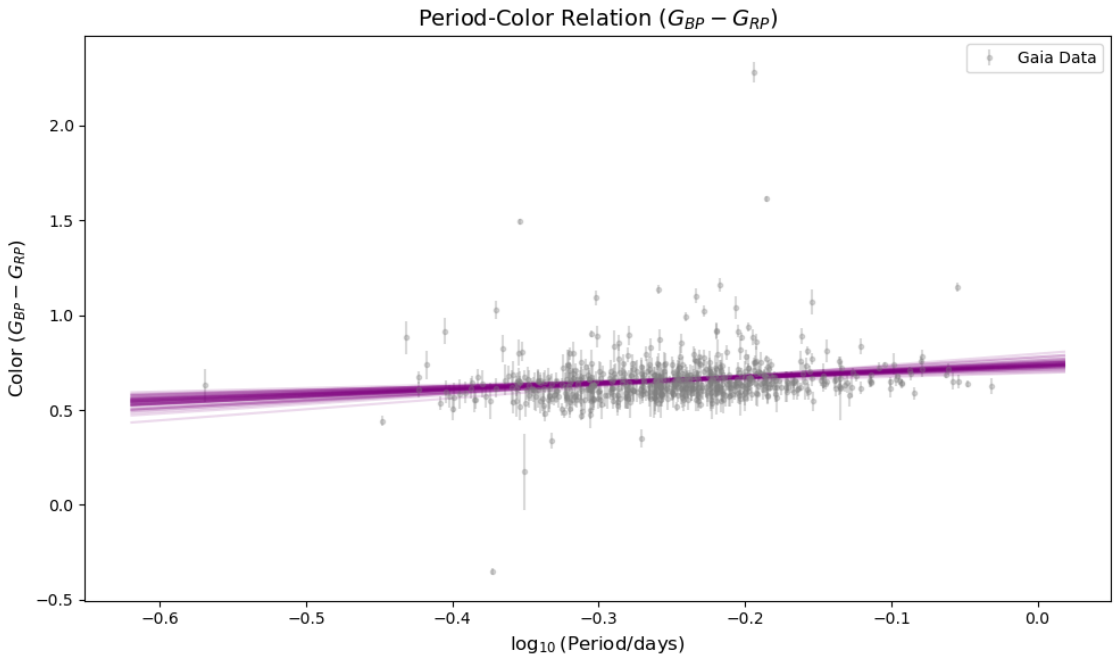
```
err_arr = np.asarray(color_err, dtype=float)

# PYMC MODEL PERIOD-COLOR
print("Setting up Period-Color PyMC Model...")
with pm.Model() as model_pc:
    # Colors are much smaller than absolute magnitudes, so adjusting the bounds
    a = pm.Uniform('a', lower=-5.0, upper=5.0, initval=0.0)
    b_centered = pm.Uniform('b_centered', lower=-2.0, upper=2.0, initval=0.5)
    sigma_scatter = pm.Uniform('sigma_scatter', lower=0.0, upper=1.0, initval=0.
 ↪05)

    x_pt = pt.as_tensor_variable(x_arr)
    y_pt = pt.as_tensor_variable(y_arr)
    err_pt = pt.as_tensor_variable(err_arr)

    # Model
    mu = a * x_pt + b_centered
    sigma_total = pt.sqrt(err_pt**2 + sigma_scatter**2)

    # Likelihood
    y_obs = pm.Normal('y_obs', mu=mu, sigma=sigma_total, observed=y_pt)

    # Transform back
    b_original = pm.Deterministic('b_original', b_centered - a * mean_logP)

    print("Sampling NUTS...")
    trace_pc = pm.sample(
        draws=2000,
        tune=1000,
        chains=1,
        compute_convergence_checks=False,
        return_inferencedata=False,
        progressbar=True
    )

# plots
print("Generating Plots...")

# Helper function to extract chains
def get_chain_array(trace):
    vars_to_plot = ['a', 'b_original', 'sigma_scatter']
    extracted = []
    for var in vars_to_plot:
        if hasattr(trace, 'posterior'):
            data = trace.posterior[var].values.flatten()
        else:
            data = trace.get_values(var, combine=True)
```

```
        extracted.append(data)
    return np.vstack(extracted).T

chain_pc = get_chain_array(trace_pc)

# Corner Plot
fig_corner = corner.corner(
    chain_pc,
    color='purple',
    labels=['$a$ (Slope)', '$b$ (Intercept)', r'$\sigma_{scatter}$'],
    show_titles=True,
    title_kwargs={"fontsize": 12}
)
plt.suptitle("Period-Color Relation Corner Plot", y=1.02, fontsize=14)
plt.show()

# Posterior Predictive Check Plot
fig, ax = plt.subplots(figsize=(10, 6))
ax.errorbar(logP, color, yerr=color_err, fmt='o', color='gray', alpha=0.3,␣
 ↪markersize=3, label='Gaia Data')

# Pick 50 random steps
random_indices = np.random.randint(0, len(chain_pc), 50)
x_vals = np.linspace(min(logP)-0.05, max(logP)+0.05, 100)

for idx in random_indices:
    a_samp, b_samp, _ = chain_pc[idx]
    ax.plot(x_vals, a_samp * x_vals + b_samp, color='purple', alpha=0.15)

ax.set_title('Period-Color Relation ($G_{BP} - G_{RP}$)', fontsize=14)
ax.set_xlabel('$\log_{10}(\mathrm{Period / days})$', fontsize=12)
ax.set_ylabel('Color ($G_{BP} - G_{RP}$)', fontsize=12)
ax.legend()
plt.tight_layout()
plt.show()
```

```
Initializing NUTS using jitter+adapt_diag…

Setting up Period-Color PyMC Model…
Sampling NUTS…

Sequential sampling (1 chains in 1 job)
NUTS: [a, b_centered, sigma_scatter]

Output()


Sampling 1 chain for 1_000 tune and 2_000 draw iterations (1_000 + 2_000 draws
total) took 601 seconds.
```

```
Generating Plots…
```



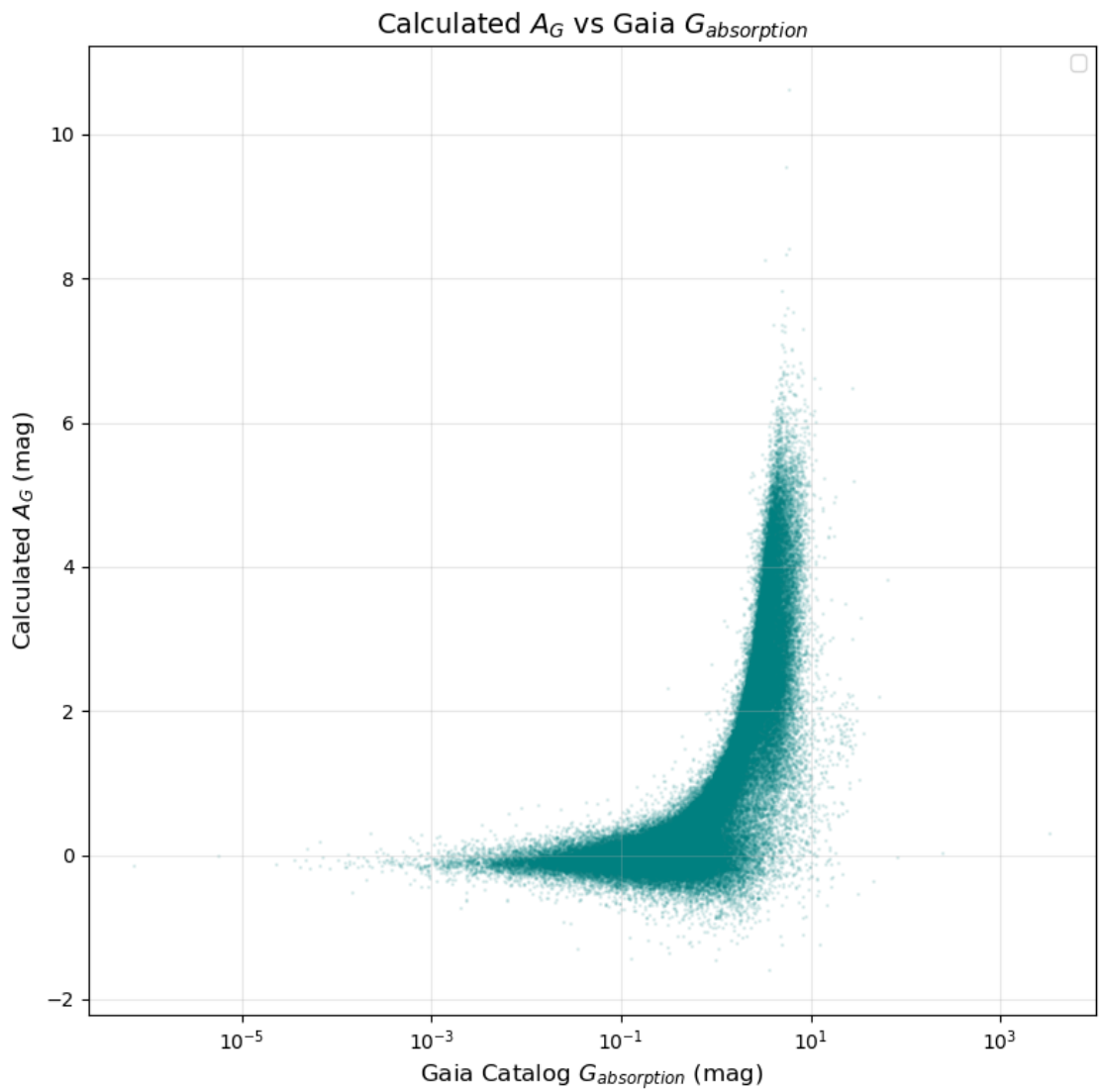


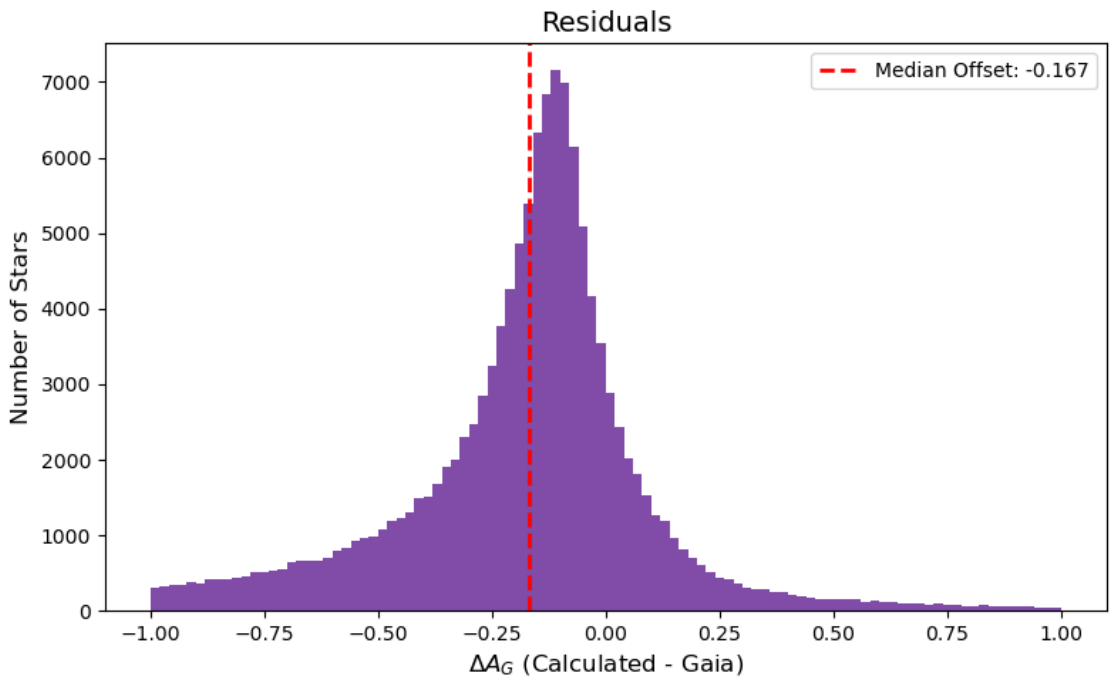
[25]:
```
import numpy as np
import pandas as pd
from astroquery.gaia import Gaia


query_full = """
SELECT
    s.source_id, s.l, s.b, s.ra, s.dec,
    s.phot_g_mean_mag, s.bp_rp, v.g_absorption,
    v.pf AS period, s.phot_g_mean_flux_over_error, s.phot_bp_rp_excess_factor
FROM gaiadr3.vari_rrlyrae AS v
JOIN gaiadr3.gaia_source AS s
    ON v.source_id = s.source_id
WHERE v.pf IS NOT NULL
  AND s.bp_rp IS NOT NULL
"""

# Use async for massive queries to prevent timeouts
job = Gaia.launch_job_async(query_full)
df_full = job.get_results().to_pandas()


# get the median parameters from the PyMC trace
# chain_pc[:, 0] is the slope, chain_pc[:, 1] is the original intercept
a_median = np.median(chain_pc[:, 0])
```

```
b_median = np.median(chain_pc[:, 1])


# Calculate the Intrinsic Color for every star based on its period
logP_full = np.log10(df_full['period'].values)
intrinsic_color = a_median * logP_full + b_median

# Calculate Color Excess: E(BP - RP) = Observed - Intrinsic
observed_color = df_full['bp_rp'].values
color_excess = observed_color - intrinsic_color

# Calculate G-band Extinction: A_G = R_G * E(BP - RP)
R_G = 2.0
A_G = R_G * color_excess
df_full['E_BP_RP'] = color_excess
df_full['A_G'] = A_G


valid_comparison_mask = np.isfinite(df_full['g_absorption']) & np.
 ↪isfinite(df_full['A_G'])
gaia_ag = df_full['g_absorption'][valid_comparison_mask]
our_ag = df_full['A_G'][valid_comparison_mask]

# Calculate the numerical residuals (Calculated values - Gaia's values)
residuals = our_ag - gaia_ag
median_offset = np.median(residuals)
std_offset = np.std(residuals)

print(f"Number of stars with Gaia g_absorption available: {len(gaia_ag):,}")
print(f"Median Offset (A_G - Gaia A_G) = {median_offset:.3f} mag")
print(f"Standard Deviation of Offset = {std_offset:.3f} mag")

plt.figure(figsize=(8, 8))
plt.scatter(gaia_ag, our_ag, s=1, alpha=0.1, color='teal')

plt.xlabel('Gaia Catalog $G_{absorption}$ (mag)', fontsize=12)
plt.ylabel('Calculated $A_G$ (mag)', fontsize=12)
plt.title('Calculated $A_G$ vs Gaia $G_{absorption}$', fontsize=14)
plt.xscale('log')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# Histogram of Residuals
plt.figure(figsize=(8, 5))
plt.hist(residuals, bins=100, range=(-1, 1), color='indigo', alpha=0.7)
```

```
plt.axvline(median_offset, color='red', linestyle='dashed', linewidth=2,␣
 ↪label=f'Median Offset: {median_offset:.3f}')
plt.xlabel('$\Delta A_G$ (Calculated - Gaia)', fontsize=12)
plt.ylabel('Number of Stars', fontsize=12)
plt.title('Residuals', fontsize=14)
plt.legend()
plt.tight_layout()
plt.show()
```

```
INFO: Query finished. [astroquery.utils.tap.core]
Number of stars with Gaia g_absorption available: 142,867
Median Offset (A_G - Gaia A_G) = -0.167 mag
Standard Deviation of Offset = 8.997 mag
```



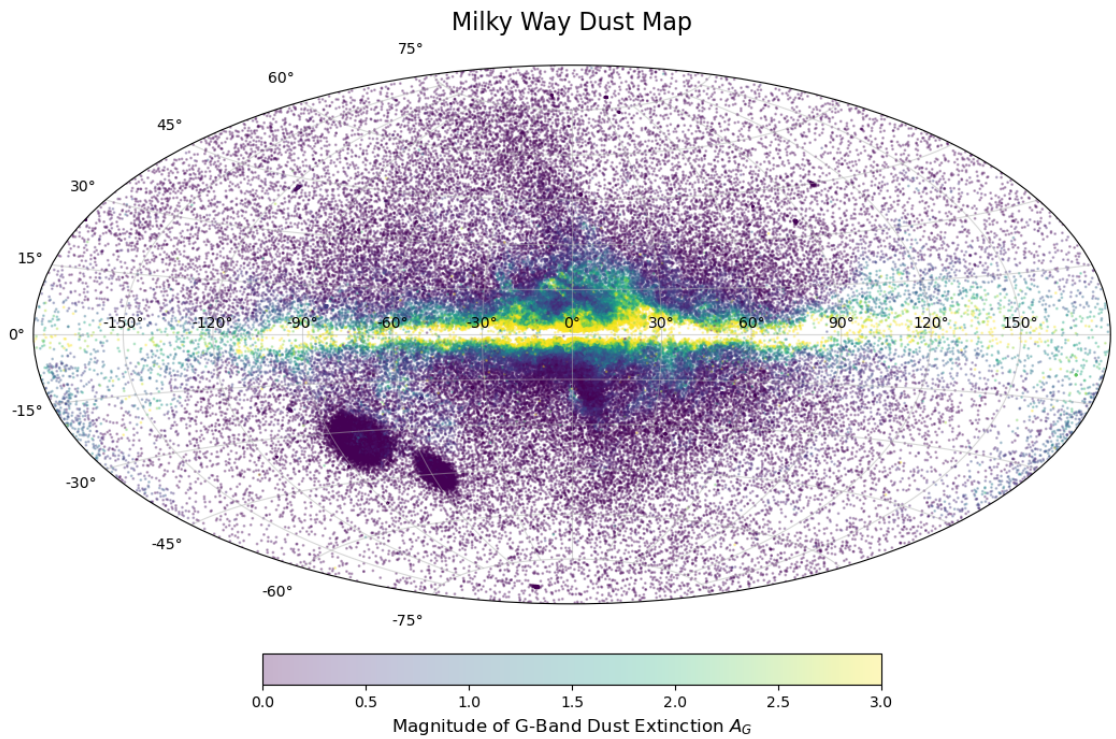


[26]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from astroquery.gaia import Gaia

# query
print("Launching ADQL query for the map (including SNR and BP/RP excess)...")
query_map = """
SELECT
    s.source_id, s.l, s.b,
    s.bp_rp,
    v.pf AS period,
    s.phot_g_mean_flux_over_error AS g_snr,
    s.phot_bp_rp_excess_factor AS bp_rp_excess
FROM gaiadr3.vari_rrlyrae AS v
JOIN gaiadr3.gaia_source AS s
    ON v.source_id = s.source_id
WHERE v.pf IS NOT NULL
  AND s.bp_rp IS NOT NULL
"""

job = Gaia.launch_job_async(query_map)
df_map = job.get_results().to_pandas()
print(f"Downloaded {len(df_map):,} total RR Lyrae stars.")
```

```python
# calculating dust extinction (A_G)
# I used a try/except block to  grab the median parameters from my PyMC trace
try:
    a_med = np.median(chain_pc[:, 0])
    b_med = np.median(chain_pc[:, 1])
except NameError:
    print("chain_pc not found in memory. Using standard fallback RR Lyrae␣
 ↪relations.")
    a_med = -1.5
    b_med = 0.5

df_map['intrinsic_color'] = a_med * np.log10(df_map['period']) + b_med
df_map['color_excess'] = df_map['bp_rp'] - df_map['intrinsic_color']
df_map['A_G'] = 2.0 * df_map['color_excess']

# Cleaning data
# SN Ratio cut (> 20)
# limiting BP/RP light scattering excess (< 1.5)
# remove negative extinctions
mask = (df_map['g_snr'] > 20) & \
       (df_map['bp_rp_excess'] < 1.5) & \
       (df_map['A_G'] > -0.5)

df_clean_map = df_map[mask]

print(f"Stars remaining: {len(df_clean_map):,}")

#plot
# converting to radians since im using aitoff
b_rad = np.radians(df_clean_map['b'])

# Longitude must go from -pi to pi
l_rad = np.radians(df_clean_map['l'])
l_rad = np.where(l_rad > np.pi, l_rad - 2*np.pi, l_rad)

ag_vals = df_clean_map['A_G']
fig = plt.figure(figsize=(12, 7))
ax = fig.add_subplot(111, projection='aitoff')

# Scatter the stars.
# s=1 makes points tiny, alpha=0.3 makes them semi-transparent
# capped vmax=3.0 so the extreme outliers don't wash out the color scale
sc = ax.scatter(l_rad, b_rad, c=ag_vals, cmap='viridis', s=1, alpha=0.3,␣
 ↪vmin=0, vmax=3.0)

ax.grid(True, alpha=0.5)
```

```
cbar = plt.colorbar(sc, orientation='horizontal', fraction=0.05, pad=0.08)
cbar.set_label('Magnitude of G-Band Dust Extinction $A_G$', fontsize=12)

ax.set_title("Milky Way Dust Map", fontsize=16, y=1.05)
plt.tight_layout()
plt.show()
```

```
Launching ADQL query for the map (including SNR and BP/RP excess)…
INFO: Query finished. [astroquery.utils.tap.core]
Downloaded 170,107 total RR Lyrae stars.
Stars remaining: 129,969
```



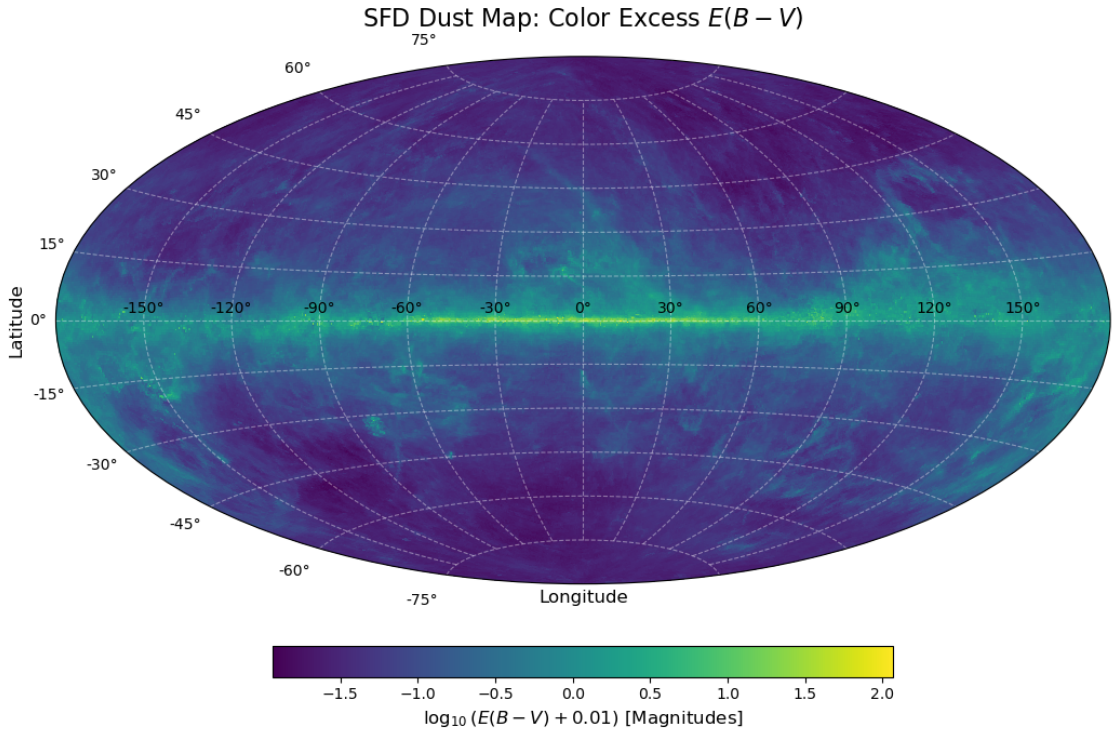
The distribution has areas that clearly have more observations that others. The obvious one that I just put all this work into finding is due to the dust located at the center plane of the interstellar medium. Another reason why there could be more stars in some areas is due to the way Gaia makes observations. Gaia observes different patches of the sky, favoring some areas more than others. Since stars need multiple good observations before they can be added to the catalog, the stars that lie in the regions that get more attention are more likely to be added to the data base. As a result we observe some very dense patches of RR Lyrae, like the ones between -30 and -50 degrees latitude.

[27]:
```
import numpy as np
import matplotlib.pyplot as plt
```

```python
from astropy.coordinates import SkyCoord
import astropy.units as u
from dustmaps.sfd import SFDQuery
import dustmaps.sfd

dustmaps.sfd.fetch()

# creating plotting space
l_grid = np.linspace(-np.pi, np.pi, 800)
b_grid = np.linspace(-np.pi/2., np.pi/2., 400)
L, B = np.meshgrid(l_grid, b_grid)

coords = SkyCoord(l=L*u.rad, b=B*u.rad, frame='galactic') # Converting the grid␣
↪to Astropy SkyCoords


# SFD query
print("Querying the dust map grid...")
sfd = SFDQuery()
ebv = sfd(coords)

# aitoff plot
fig = plt.figure(figsize=(12, 7))
ax = fig.add_subplot(111, projection='aitoff')

# I plotted the Log10 of the dust because the galactic center is WAY dustier␣
↪than the poles. The log scale makes the faint structures visible. (Adding 0.
↪01 prevents taking the log of true zero)
im = ax.pcolormesh(L, B, np.log10(ebv + 0.01), cmap='viridis', shading='auto')

# Formatting
ax.set_title('SFD Dust Map: Color Excess $E(B-V)$', pad=20, fontsize=16)
ax.set_xlabel('Longitude', fontsize=12)
ax.set_ylabel('Latitude', fontsize=12)
ax.grid(True, alpha=0.4, color='white', linestyle='--')

# Add a colorbar
cbar = fig.colorbar(im, orientation='horizontal', fraction=0.05, pad=0.1)
cbar.set_label('$\log_{10}(E(B-V) + 0.01)$ [Magnitudes]', fontsize=12)

plt.tight_layout()
plt.show()
```

```
Downloading SFD data file to /Users/hjugan/Desktop/School/2026/Spring/ASTRON
128/astro_128_labs/lab1/dustmaps_data/sfd/SFD_dust_4096_ngp.fits
Checking existing file to see if MD5 sum matches …
File exists. Not overwriting.
```

```
Downloading SFD data file to /Users/hjugan/Desktop/School/2026/Spring/ASTRON
128/astro_128_labs/lab1/dustmaps_data/sfd/SFD_dust_4096_sgp.fits
Checking existing file to see if MD5 sum matches …
File exists. Not overwriting.
Querying the dust map grid…
```



Macroscopically my map is near identical to the SFD dust map. They both have a large band in the center that is generated from large concentrations of interstellar dust. However, I would not expect my map to look EXACTLY like the SFD dust map. The primary difference between the SFD map and mine is the continuity between points. Since my map was generated using RR Lyrae stars, there are a lot of blank patches where no RR Lyrae stars were in the sky. Since the SFD doesn't rely on only RR Lyrae's it is fully filled in with no blank patches. This also characterizes a limitation, the lack of continuous data points can make my map unreliable across the entire sky.

[ ]:

```

```